VIV-32 Architecture Specification v0.2

Viveretur

September 3, 2026

Contents

1 Overview **7**
- 1.1 Design Goals . . . 7
- 1.2 Goals for VIV-32 v0.2 . . . 7
- 1.3 Non-Goals for VIV-32 v0.2 . . . 7
- 1.4 Specification Status . . . 8

2 Programming Model **9**
- 2.1 Architecture Identity . . . 9
- 2.2 Word and Address Model . . . 9
- 2.3 Instruction Model . . . 9
- 2.4 Register Model . . . 10
- 2.5 Arithmetic and Flags . . . 11
- 2.6 Memory Access . . . 12
- 2.7 Control Flow . . . 13
- 2.8 Reset, Exceptions and Interrupts . . . 13

3 Memory Model **15**
- 3.1 Address Space . . . 15
- 3.2 Endianness . . . 15
- 3.3 Access Sizes . . . 15
- 3.4 Alignment . . . 15
- 3.5 Instruction Alignment . . . 15
- 3.6 Memory-Mapped I/O . . . 16
- 3.7 Caching . . . 16
- 3.8 Atomicity . . . 16
- 3.9 Memory Ordering . . . 16

4 Instruction Encoding **17**
- 4.1 Instruction Word . . . 17
- 4.2 Major Opcode Field . . . 17
- 4.3 Register Fields . . . 18
- 4.4 Instruction Format Families . . . 18
  - 4.4.1 R-Type: Register-Register Operations . . . 18
  - 4.4.2 R2-Type: Register-Register Operations with Two Destinations . . . 19
  - 4.4.3 I-Type: Register-Immediate Operations . . . 20
  - 4.4.4 BI-Type: Register-Immediate Bit Operations . . . 20
  - 4.4.5 U-Type: Upper Immediate and Constant Construction . . . 21
  - 4.4.6 M-Type: Load and Store Operations . . . 21
  - 4.4.7 BF-Type: Flag-Based Conditional Branches . . . 23
  - 4.4.8 BC-Type: Register-Comparison Branches . . . 24
  - 4.4.9 J-Type: PC-Relative Jump and Call . . . 25

4.4.10 JR-Type: Register-Indirect Jump and Call . . . 26
4.4.11 X-Type: System and Control Operations . . . 26
4.5 Major Opcode Map . . . 27
4.6 Class-Specific Encoding Values . . . 28
4.6.1 Register ALU Function Codes . . . 28
4.6.2 Immediate Arithmetic and Compare Mode Codes . . . 29
4.6.3 Immediate Logical Mode Codes . . . 29
4.6.4 Register Shift Function Codes . . . 29
4.6.5 Immediate Shift Mode Codes . . . 30
4.6.6 Bit Immediate Mode Codes . . . 30
4.6.7 Multiply and Divide Function Codes . . . 30
4.6.8 Constant Construction Mode Codes . . . 31
4.7 Memory Encoding Values . . . 31
4.7.1 Load Encoding . . . 31
4.7.2 Store Encoding . . . 31
4.7.3 Memory Access Size Field . . . 32
4.8 Inter-File Transfer . . . 32
4.9 Branch Condition Codes . . . 33
4.9.1 Flag-Based Branch Conditions . . . 33
4.9.2 Register-Comparison Branch Conditions . . . 33
4.10 Jump and Call Encoding Values . . . 34
4.10.1 PC-Relative Jump and Call Opcodes . . . 34
4.10.2 Register-Indirect Jump and Call Function Codes . . . 34
4.11 System and Control Encoding Values . . . 34
4.11.1 System Operation Codes . . . 35
4.11.2 System Payload Encoding . . . 35
4.11.3 System Instruction Register Fields . . . 35
4.11.4 Control Register Numbers . . . 36
4.12 NOP Encoding . . . 36
4.13 Flag Update Policy . . . 36
4.14 Reserved Encodings . . . 36

5 Instruction Set **37**
5.1 Notation . . . 37
5.1.1 Instruction Notation . . . 37
5.1.2 Assignment Notation . . . 38
5.1.3 Immediate Values . . . 38
5.1.4 Conditional Suffixes . . . 38
5.2 Flag Behaviour . . . 39
5.2.1 Negative Flag . . . 39
5.2.2 Zero Flag . . . 39
5.2.3 Carry Flag . . . 40
5.2.4 Overflow Flag . . . 40
5.2.5 Arithmetic Error Flag . . . 40
5.2.6 Infinity Flag . . . 40
5.2.7 Underflow Flag . . . 41
5.2.8 Inexact Flag . . . 41
5.3 Arithmetic Instructions . . . 41
5.3.1 `add, addd` . . . 41
5.3.2 `addi` . . . 41
5.3.3 `sub, subd` . . . 41
5.3.4 `subi` . . . 41

5.3.5 **neg, negd** . . . 41
5.4 Logical Instructions . . . 42
5.4.1 **and, andd** . . . 42
5.4.2 **andi** . . . 42
5.4.3 **or, ord** . . . 42
5.4.4 **ori** . . . 42
5.4.5 **xor, xord** . . . 42
5.4.6 **xori** . . . 42
5.4.7 **not, notd** . . . 42
5.5 Comparison Instructions . . . 43
5.5.1 **cmp, cmpd** . . . 43
5.5.2 **cmpi** . . . 43
5.5.3 btst, btstd . . . 43
5.6 Shift Instructions . . . 44
5.6.1 Shift Register Behaviour . . . 44
5.6.2 **shl, shld** . . . 44
5.6.3 **shli, shldi** . . . 44
5.6.4 **shr, shrd** . . . 44
5.6.5 **shri, shrdi** . . . 45
5.6.6 **sar, sard** . . . 45
5.6.7 **sari, sardi** . . . 45
5.7 Bit Manipulation Instructions . . . 45
5.7.1 **bset, bsetd** . . . 45
5.7.2 **bclr, bclrd** . . . 45
5.7.3 **btgl, btgld** . . . 45
5.8 Multiply and Divide Instructions . . . 46
5.8.1 **mul** . . . 46
5.8.2 **muld** . . . 46
5.8.3 **mulu** . . . 46
5.8.4 **div** . . . 47
5.8.5 **divd** . . . 47
5.8.6 **divu** . . . 48
5.9 Load and Store Instructions . . . 48
5.9.1 Address Calculation . . . 48
5.9.2 Alignment . . . 49
5.9.3 Flag Behaviour . . . 49
5.9.4 **lb** . . . 49
5.9.5 **lbu** . . . 49
5.9.6 **lh** . . . 50
5.9.7 **lhu** . . . 50
5.9.8 **lw** . . . 50
5.9.9 **ld** . . . 50
5.9.10 **ldl** . . . 50
5.9.11 **ldu** . . . 50
5.9.12 **sb** . . . 51
5.9.13 **sh** . . . 51
5.9.14 **sw** . . . 51
5.9.15 **sd** . . . 51
5.9.16 **sdl** . . . 51
5.9.17 **sdu** . . . 51
5.10 Constant Construction Instructions . . . 52

5.10.1 `lui` . . . 52
5.10.2 `lli` . . . 52
5.10.3 `lhi` . . . 52
5.10.4 Full-Width Constants, `li` and `la` . . . 52
5.11 Inter-File Transfer Instructions . . . 52
5.11.1 Floating-Point Rounding . . . 52
5.11.2 `disf` . . . 53
5.11.3 `utod` . . . 53
5.11.4 `itod` . . . 53
5.11.5 `ftod` . . . 53
5.11.6 `dtou` . . . 54
5.11.7 `dtoi` . . . 54
5.11.8 `dtof` . . . 55
5.12 Branch Instructions . . . 55
5.12.1 Branch Target Calculation . . . 55
5.12.2 Register-Comparison Branches . . . 55
5.12.3 `b.eq, bd.eq` . . . 56
5.12.4 `b.ne, bd.ne` . . . 56
5.12.5 `b.lt, bd.lt` . . . 56
5.12.6 `b.le, bd.le` . . . 56
5.12.7 `b.gt, bd.gt` . . . 56
5.12.8 `b.ge, bd.ge` . . . 56
5.12.9 `b.ltu` . . . 56
5.12.10 `b.leu` . . . 57
5.12.11 `b.gtu` . . . 57
5.12.12 `b.geu` . . . 57
5.12.13 Flag-Based Branches . . . 57
5.12.14 `bf.ns` . . . 57
5.12.15 `bf.nc` . . . 57
5.12.16 `bf.zs` . . . 57
5.12.17 `bf.zc` . . . 57
5.12.18 `bf.cs` . . . 57
5.12.19 `bf.cc` . . . 58
5.12.20 `bf.vs` . . . 58
5.12.21 `bf.vc` . . . 58
5.12.22 `bf.es` . . . 58
5.12.23 `bf.ec` . . . 58
5.12.24 `bf.is` . . . 58
5.12.25 `bf.ic` . . . 58
5.12.26 `bf.us` . . . 58
5.12.27 `bf.uc` . . . 58
5.12.28 `bf.xs` . . . 58
5.12.29 `bf.xc` . . . 59
5.13 Jump and Call Instructions . . . 59
5.13.1 Jump Target Rules . . . 59
5.13.2 `jmp` . . . 59
5.13.3 `call` . . . 59
5.13.4 `jr` . . . 59
5.13.5 `jalr` . . . 60
5.13.6 `ret` . . . 60
5.13.7 Nested Calls . . . 60

5.14 System and Control Instructions . . . 60
5.14.1 Control Registers . . . 60
5.14.2 `nop` . . . 61
5.14.3 `halt` . . . 61
5.14.4 `trap` . . . 61
5.14.5 `syscall` . . . 62
5.14.6 iret . . . 62
5.14.7 `ei` . . . 62
5.14.8 `di` . . . 62
5.14.9 `rdpc` . . . 63
5.14.10 `mrs` . . . 63
5.14.11 `msr` . . . 63
5.15 Pseudo-Instructions . . . 63
5.15.1 Pseudo-Instruction Policy . . . 64
5.15.2 `ldd` . . . 64
5.15.3 `sdd` . . . 64
5.15.4 `li` . . . 64
5.15.5 `la` . . . 64
5.15.6 `mov, movd` . . . 65
5.15.7 `clr, clrd` . . . 65
5.15.8 `ret` . . . 65
5.15.9 `push, pushd` . . . 65
5.15.10 `pop, popd` . . . 65
5.16 Reserved Instructions . . . 65
5.17 Instruction Set Status . . . 66

6 Exceptions and Interrupts **67**
6.1 Exception and Interrupt Model . . . 67
6.2 Exception Causes . . . 67
6.3 Exception Vector Calculation . . . 68
6.4 Exception Entry Sequence . . . 68
6.5 Exception Program Counter . . . 68
6.6 Exception Data Register . . . 69
6.7 Status Register Interrupt State . . . 69
6.8 Interrupts . . . 70
6.9 Software Trap . . . 70
6.10 System Call . . . 70
6.11 Illegal Instruction . . . 71
6.12 Misaligned Access Exceptions . . . 71
6.13 Bus Error . . . 71
6.14 Exception Return . . . 72
6.15 Nested Exceptions and Interrupts . . . 72

7 Reset and Boot **73**
7.1 Reset Model . . . 73
7.2 Reset Vector . . . 73
7.3 Reset State . . . 73
7.4 Reset Cause . . . 74
7.5 Initial Interrupt State . . . 74
7.6 Initial Memory State . . . 74
7.7 Boot Responsibilities . . . 75
7.8 Boot Sequence Example . . . 75

8 Implementation Notes **76**

Chapter 1

Overview

VIV-32 is a 32-bit load/store architecture designed for experimentation, education, and implementation across multiple execution targets. The architecture is intended to support a reference emulator, a QEMU target, and a SystemVerilog or Verilog hardware implementation.

The initial version of the architecture prioritises regular instruction decoding, straightforward toolchain development, and simple hardware implementation over instruction density or microarchitectural sophistication.

1.1 Design Goals

- Provide a clean target for assemblers, compilers, interpreters, and firmware.
- Keep the base ISA simple enough to implement in both software and hardware.
- Support a flat 32-bit physical address space.
- Support memory-mapped I/O.
- Support timer interrupts for a future supervising operating environment.
- Provide enough structure for an ABI, assembler, linker, and emulator.
- Preserve room for future extensions such as atomics, SIMD, vector, matrix, or co-processor features.

1.2 Goals for VIV-32 v0.2

- Implement floating-point support.

1.3 Non-Goals for VIV-32 v0.2

- Virtual memory.
- Paging.
- Segmentation.
- Cache architecture.
- Multiprocessing memory coherency.
- Dynamic linking.

- User/supervisor privilege separation.
- Binary compatibility with an existing architecture.

1.4 Specification Status

- This document describes the architectural behaviour of VIV-32.
- The ABI is specified separately in the VIV-32 ABI Specification.
- Platform devices and concrete memory maps are specified separately in the VIV-32 Platform Specification.
- Toolchain object formats, assembler syntax, and linker behaviour are specified separately in the VIV-32 Toolchain Specification.

Chapter 2

Programming Model

2.1 Architecture Identity

- Architecture name: VIV-32.
- Toolchain and implementation prefix: `viv32`.
- Architecture style: 32-bit load/store architecture.
- Primary implementation targets:
  - reference emulator;
  - QEMU target;
  - SystemVerilog/Verilog hardware implementation.

2.2 Word and Address Model

- Word size: 32 bits.
- Address size: 32 bits.
- Byte size: 8 bits.
- Endianness: big-endian.
- Memory model: flat byte-addressed memory.
- Address space model: flat 32-bit physical address space.
- Virtual memory / paging: not supported in VIV-32 v0.2.
- Segmentation: not supported in VIV-32 v0.2.

2.3 Instruction Model

- Instruction width: fixed 32-bit instructions.
- Instruction encoding style: regular RISC-style encoding with a small number of instruction format families.
- Variable-width instructions: not supported in VIV-32 v0.2.

- Compressed or variable-width instruction encodings are deferred to a later architecture extension.
- Separate register tables for general purpose and floating point operations.
- Load/store architecture: yes.
- Arithmetic and logical operations operate on registers.
- Memory is accessed only through explicit load and store instructions.
- Memory operand policy: arithmetic and logical instructions do not operate directly on memory operands.
- Immediate sizes: defined per instruction format.
- Branch offset unit: 32-bit instruction words, shifted left by two bits to form byte offsets.
- Branch range: BF-type branches use `offset22`; BC-type branches use `offset14`.
- Opcode extension policy: reserve opcode space for future architectural extensions.
- Possible future extensions include:
  - compressed instructions;
  - atomic instructions;
  - privileged memory-management features;
  - SIMD, vector, matrix, or co-processor operations.

2.4 Register Model

- Sixteen 32-bit general-purpose registers with signed and unsigned operations.
- Eight 64-bit floating-point registers.
- General-purpose registers are named `$0` through `$15`.
- Floating-point registers are named `#0` through `#7`.
- Register `$0` is hardwired to zero; reads always return zero, and writes are ignored.
- Register `$13` is reserved by ABI convention as an optional frame pointer `$fp`.
- Register `$14` is reserved by ABI convention as the stack pointer `$sp`.
- Register `$15` is used as the link register `$lr` for call and return instructions.
- The frame pointer is not a dedicated architectural register.
- The program counter is directly readable using a dedicated instruction.
- The program counter is not directly writable as a general-purpose register.
- Control flow changes are performed through:
  - branch instructions;
  - jump instructions;
  - call instructions;

  - return instructions;
  - trap instructions;
  - interrupt-return instructions.

- Special/control registers include:
  - `%pc`: program counter;
  - `%sr`: status register;
  - `%epc`: exception program counter;
  - `%esr`: exception status register, containing the status-register value saved on exception entry;
  - `%ecause`: exception cause;
  - `%edata`: exception-specific data associated with the most recent exception or interrupt.
  - `%evbase`: exception vector base.

- Parallel/vector register support is not included in the base ISA.
- Future extensions may define SIMD, vector, matrix, or co-processor facilities.
- Matrix or co-processor support is deferred to a future extension or memory-mapped accelerator design.

2.5 Arithmetic and Flags

- Integer representation: two's complement.
- Floating-point register representation: IEEE-754 binary64.
- Floating-point memory representation: IEEE-754 binary32 and binary64.
- Signed and unsigned integer arithmetic results wrap modulo 2^{32}.
- Floating-point arithmetic is performed using IEEE-754 binary64 precision.
- Floating-point arithmetic and floating-point conversions use round-to-nearest, ties-to-even unless otherwise specified by an instruction.
- VIV-32 defines condition flags in the processor status register.
- Condition flags are `NZCVEIUX`:
  - `NEGATIVE` (`N`): set when the numeric result of a flag-setting operation is negative. For floating-point results, the sign of zero does not cause `N` to be set.
  - `ZERO` (`Z`): set when the numeric result of a flag-setting operation is zero. For floating-point operations, both `+0.0` and `-0.0` are considered zero.
  - `CARRY` (`C`): used by integer arithmetic. Set when an unsigned addition produces a carry out of bit 31. For subtraction, `C` is set when no borrow is required and cleared when a borrow is required. Floating-point operations clear `C` unless otherwise specified.
  - `OVERFLOW` (`V`): used by integer arithmetic. Set when the mathematical result cannot be represented in the destination integer domain. Floating-point operations clear `V` unless otherwise specified.

  - `ARITHMETIC ERROR` (`E`): set when an arithmetic or conversion operation encounters an exceptional or invalid condition that does not trap. This includes integer divide-by-zero, floating-point divide-by-zero, invalid floating-point operations, unordered flag-setting floating-point comparisons, and invalid numeric conversions.
  - `INFINITY` (`I`): used by floating-point operations. Set when the resulting IEEE-754 value is positive or negative infinity. Integer operations clear `I` unless otherwise specified.
  - `UNDERFLOW` (`U`): used by floating-point operations. Set when a floating-point result is too small in magnitude to be represented normally in the destination floating-point format. Integer operations clear `U` unless otherwise specified.
  - `INEXACT` (`X`): used by floating-point operations and conversions. Set when the mathematically exact result cannot be represented exactly in the destination format and rounding is therefore required. Integer operations clear `X` unless otherwise specified.

- Integer arithmetic operations update the relevant `NZCVE` flags and clear `IUX` unless otherwise specified.
- Floating-point arithmetic operations update the relevant `NZEIUX` flags and clear `CV` unless otherwise specified.
- Floating-point divide-by-zero sets `E`; if the resulting value is infinity, `I` is also set.
- Invalid floating-point operations producing `NaN` set `E`.
- Flag-setting floating-point comparisons involving `NaN` set `E` to indicate an unordered comparison.
- Floating-point to integer conversion of `NaN`, infinity, or a value outside the destination integer range sets `E`.
- Floating-point narrowing from binary64 to binary32 updates `NZIUX` according to the binary32 result and sets `E` only when the conversion is otherwise invalid.
- Integer comparisons update the relevant integer comparison flags and clear floating-point-specific flags unless otherwise specified.
- Flag-setting floating-point comparisons update the relevant floating-point flags; `C` and `V` have no floating-point comparison meaning.
- Flag-based branch instructions test the current processor-status flags. They do not interpret the flags as signed, unsigned, or floating-point comparisons and do not perform a comparison themselves.

2.6 Memory Access

- Supported load sizes:
  - 8-bit byte;
  - 16-bit halfword;
  - 32-bit word;
- Supported store sizes:
  - 8-bit byte;

  - 16-bit halfword;
  - 32-bit word;
- Byte accesses may occur at any address.
- Halfword accesses must be aligned to a 2-byte boundary.
- Word accesses must be aligned to a 4-byte boundary.
- Misaligned access behaviour: misaligned halfword or word accesses raise a `MisalignedAccessException`.
- VIV-32 v0.2 defines no cache architecture.
- All memory accesses are treated as direct accesses to memory or memory-mapped devices.
- VIV-32 uses memory-mapped I/O.
- Device registers occupy reserved physical address ranges defined by the platform specification.

2.7 Control Flow

- Jump mechanism: `PC`-relative immediate jumps (`jmp`) and register-indirect jumps (`jr`).
- Call mechanism: `PC`-relative call with link register `call`, plus register-indirect call (`jalr`).
- Link register behaviour: `call` instructions store the address of the next instruction in `%lr`. `jalr` provides the option to use a different return register.
- Return mechanism: `ret` jumps to the address held in the link register `%lr`. A general register-indirect jump instruction may also be used for returns, if previously loaded with an address via `jalr`.
- `trap` and `syscall` instructions transfer control to their associated exception vectors and record the continuation address and cause in exception control registers `%epc` and `%ecause`. `trap` additionally stores its immediate operand in `%edata`. `syscall` sets `%edata` to `0`.
- Halt instruction behaviour: the `halt` instruction stops instruction execution until reset or platform-defined external action.
- NOP encoding: `0x00000000`.
- Extended immediates: VIV-32 v0.2 does not use implicit next-word operands in the base instruction stream. Full-width constants and absolute addresses can be constructed using the pseudo-instructions `li` and `la`, respectively.

2.8 Reset, Exceptions and Interrupts

- Privilege model: VIV-32 v0.2 defines machine mode only. User and supervisor modes are reserved for future versions.
- Reset: reset begins directly at `0x00`.
- Exception vector model: VIV-32 uses a vector base register `%evbase` and cause-specific vector offsets. The exact vector layout is defined later in this document.

- Interrupt vector model: interrupts transfer control through the exception vector mechanism.
- Exception types:
  - `Reset`;
  - `IllegalInstructionException`;
  - `MisalignedFetchException`;
  - `MisalignedAccessException`;
  - `SoftwareTrap`;
  - `SystemCall`;
  - `Interrupt`;
  - `BusError`.
- Interrupt enable/disable mechanism: the processor status register contains an interrupt-enable bit, which can be cleared (disabled) with `di` and set (enabled) with `ei`. The CPU starts with this bit cleared—a program wishing to use interrupts should enable this after configuring the relevant peripheral settings.
- `iret` instruction: VIV-32 defines an interrupt-return instruction which resumes execution from the saved exception program counter in `%epc`.
- Illegal instruction behaviour: illegal instructions raise an `IllegalInstructionException`.
- External interrupt: reserved for platform devices.
- Timer interrupt: supported in VIV-32 v0.2 through the general platform device interrupt ISR.
- Divide-by-zero behaviour: divide-by-zero does not trap in VIV-32 v0.2. Software is responsible for checking `ARITHMETIC ERROR`.

Chapter 3

Memory Model

3.1 Address Space

- VIV-32 uses a flat, byte-addressed 32-bit physical address space. Addresses are 32-bit unsigned values, and each byte is 8 bits.
- VIV-32 v0.2 does not support virtual memory, segmentation or paging.

3.2 Endianness

- VIV-32 is big-endian. Multi-byte values are stored with the most significant byte at the lowest memory address. A 32-bit word value `0x12345678` stored at address `A` is represented as:

Address	Byte value
`A + 0`	`0x12`
`A + 1`	`0x34`
`A + 2`	`0x56`
`A + 3`	`0x78`

3.3 Access Sizes

VIV-32 supports 8-bit byte, 16-bit halfword and 32-bit word loads and stores.

3.4 Alignment

- Byte accesses may occur at any address.
- Halfword accesses must be aligned to a 2-byte boundary.
- Word accesses must be aligned to a 4-byte boundary.
- Misaligned halfword or word accesses raise a `MisalignedAccessException`.

3.5 Instruction Alignment

- VIV-32 instructions are 32 bits wide.
- Instruction fetch addresses must be word-aligned.
- Branch and jump targets must be word-aligned.

- Attempting to fetch an instruction from a misaligned address raises a `MisalignedFetchException`.

3.6 Memory-Mapped I/O

- VIV-32 uses memory-mapped I/O (MMIO).
- Device registers occupy reserved physical address ranges.
- Exact device addresses are defined by the platform specification, not by the base ISA.

3.7 Caching

- VIV-32 v0.2 defines no cache architecture. All memory accesses are treated as direct accesses to memory or memory-mapped devices.
- Cache behaviour, cache coherency, and memory barriers may be possible future extensions.

3.8 Atomicity

- VIV-32 v0.2 does not define atomic read-modify-write memory instructions. Naturally aligned 8-, 16-, and 32-bit loads and stores are individually completed as single architectural memory operations.
- Multiprocessor atomicity, shared-memory locking, and inter-CPU memory ordering are deferred for future possible extension.

3.9 Memory Ordering

- A single processor observes its own memory operations in program order.
- Implementations may execute operations internally in any manner provided the architectural behaviour is equivalent to program-order execution.
- VIV-32 v0.2 defines no multiprocessor memory consistency model.
- Future versions may define memory barriers, atomic operations, and inter-processor ordering rules.

Chapter 4

Instruction Encoding

The encoding described in this chapter defines the architectural instruction encoding for VIV-32 v0.2.

4.1 Instruction Word

- VIV-32 instructions are fixed-width 32-bit words.
- Instruction fetch addresses must be 4-byte aligned.
- Instruction words are stored in memory using the architecture's big-endian byte order.
- Instruction encoding diagrams number bits from bit 31, the most significant bit, to bit 0, the least significant bit.
- Unless otherwise stated, immediate fields are sign-extended to 32 bits before use.
- Logical immediate instructions zero-extend their immediate operands.

For example, the instruction word `0x12345678` stored at address `A` is represented in memory as follows:

Address	Byte value
`A + 0`	`0x12`
`A + 1`	`0x34`
`A + 2`	`0x56`
`A + 3`	`0x78`

4.2 Major Opcode Field

- Bits 31–26 contain the major opcode.
- The major opcode is 6 bits wide.
- A 6-bit major opcode provides 64 primary opcode classes.
- Individual opcode classes may define additional function fields.
- Opcode space is reserved for future extensions.

Bits	Field	Description
31–26	opcode	Major opcode
25–0	payload	Format-specific instruction payload

4.3 Register Fields

- VIV-32 v0.2 has 16 general-purpose registers, and 8 floating-point registers.
- Register fields are 4 bits wide.
- A register field value of `0x0` selects register `$0` or register `#0`, depending on the instruction context.
- A register field value of `0xF` selects GPR register `$15`, but is invalid for the FPR as it only goes to `#7`.
- Register field names such as `rd`, `ra`, `rb`, `rd0`, `rd1`, `rs`, `base`, and `target` are documentation names for encoded register fields.
- Register fields are placed on nibble boundaries. Register fields are placed in the least-significant nibbles of the instruction word to aid in reading.

Field value	GPR	FPR
`0x0`	`$0`	`#0`
`0x1`	`$1`	`#1`
`0x2`	`$2`	`#2`
`0x3`	`$3`	`#3`
`0x4`	`$4`	`#4`
`0x5`	`$5`	`#5`
`0x6`	`$6`	`#6`
`0x7`	`$7`	`#7`
`0x8`	`$8`	-
`0x9`	`$9`	-
`0xA`	`$10`	-
`0xB`	`$11`	-
`0xC`	`$12`	-
`0xD`	`$13`	-
`0xE`	`$14`	-
`0xF`	`$15`	-

4.4 Instruction Format Families

VIV-32 v0.2 defines a small number of instruction format families. The exact opcode assignments are defined separately in the opcode map.

4.4.1 R-Type: Register-Register Operations

R-type instructions operate primarily on registers. Register fields are placed in the least-significant nibbles of the instruction word so that register operands remain readable in hexadecimal instruction dumps.

Bits	31–26	25–24	23–12	11–8	7–4	3–0
Field	`opcode`	`file`	`func/reserved`	`rd`	`ra`	`rb`

- `opcode` is the 6-bit major opcode.
- `file` is the target register file—GPR or FPR. Additional file values have been reserved for future additional register files, such as SIMD.

- **func/reserved** selects the specific register-register operation or is reserved.
- **rd** is the destination register field.
- **ra** is the first source register field.
- **rb** is the second source register field.
- Arithmetic and logical R-type instructions update condition flags by default.

Examples of R-type operations include:

- **add $rd, $ra, $rb**
- **subd #rd, #ra, #rb**
- **and $rd, $ra, $rb**
- **or $rd, $ra, $rb**
- **xord #rd, #ra, #rb**
- **itod #rd, $ra ; implicit $rb=$0**

4.4.2 R2-Type: Register-Register Operations with Two Destinations

R2-type instructions are used for operations that naturally produce two register results, such as full-width multiplication or division with remainder. The four register fields occupy the least-significant four nibbles of the instruction word.

Bits	31–26	25–24	23–16	15–12	11–8	7–4	3–0
Field	**opcode**	**file**	**func/reserved**	**rd0**	**rd1**	**ra**	**rb**

- **opcode** is the 6-bit major opcode.
- **file** is the target register file—GPR or FPR.
- **func/reserved** selects the specific two-result operation or is reserved.
- **rd0** is the first destination register field.
- **rd1** is the second destination register field. Note that for FPR instructions, this should be **0**.
- **ra** is the first source register field.
- **rb** is the second source register field.

Examples of R2-type operations include:

- **mul $rdlo, $rdhi, $ra, $rb**
- **muld #rdlo, #0, #ra, #rb**
- **divd #rdq, #0, #ra, #rb**
- **divu $rdq, $rdr, $ra, $rb**

For GPR multiplication instructions, **$rd0** receives the low 32 bits of the result and **$rd1** receives the high 32 bits of the result. For GPR division instructions, **$rd0** receives the quotient and **$rd1** receives the remainder.

4.4.3 I-Type: Register-Immediate Operations

I-type instructions operate on a GPR register and a 16-bit immediate value. Register fields are placed in the least-significant nibbles of the instruction word so that the register operands remain readable in hexadecimal dumps.

Bits	31–26	25–24	23–8	7–4	3–0
Field	`opcode`	`mode/reserved`	`imm16`	`rd`	`ra`

- `opcode` is the 6-bit major opcode.
- `mode/reserved` selects an instruction variant or is reserved.
- `imm16` is a 16-bit immediate field.
- `rd` is the destination register field.
- `ra` is the source register field.
- Unless otherwise stated, `imm16` is sign-extended to 32 bits before use.
- Logical immediate instructions zero-extend `imm16` to 32 bits before use.
- Arithmetic and logical I-type instructions update condition flags by default.

Examples of I-type operations include:

- `addi $rd, $ra, imm`
- `subi $rd, $ra, imm`
- `andi $rd, $ra, imm`
- `ori $rd, $ra, imm`
- `xori $rd, $ra, imm`
- `cmpi $ra, imm`

4.4.4 BI-Type: Register-Immediate Bit Operations

BI-type instructions operate on a register and a 14-bit immediate value. Register fields are placed in the least-significant nibbles of the instruction word so that the register operands remain readable in hexadecimal dumps.

Bits	31–26	25–24	23–22	21–8	7–4	3–0
Field	`opcode`	`mode/reserved`	`file`	`imm14`	`rd`	`ra`

- `opcode` is the 6-bit major opcode.
- `mode/reserved` selects an instruction variant or is reserved.
- `file` is the target register file—GPR or FPR.
- `imm14` is a 14-bit immediate field.
- `rd` is the destination register field.
- `ra` is the source register field.

- Arithmetic and logical BI-type instructions update condition flags by default.

Examples of BI-type operations include:

- **btst $rd, $ra, imm**
- **bsetd #rd, #ra, imm**

4.4.5 U-Type: Upper Immediate and Constant Construction

U-type instructions construct constants and addresses in general-purpose registers. U-type instructions use a 16-bit immediate field and place the destination register in the least-significant nibble of the instruction word.

Bits	31–26	25–20	19–4	3–0
Field	**opcode**	**mode/reserved**	**imm16**	**rd**

- **opcode** is the 6-bit major opcode.
- **mode/reserved** selects the constant-construction operation or is reserved.
- **imm16** is a 16-bit immediate field.
- **rd** is the destination register field.
- U-type instructions are intended to support full 32-bit constant and address construction.
- U-type immediates are interpreted according to the specific instruction.

Examples of U-type operations include:

- **lui $rd, imm16**
- **lli $rd, imm16**
- **lhi $rd, imm16**

A typical full-width constant may be constructed using a lower-immediate instruction followed by a high-immediate instruction:

```
lli  $1, 0x5678
lhi  $1, 0x1234
```

This sequence of instructions is also represented by the pseudo-instruction **li**.

4.4.6 M-Type: Load and Store Operations

M-type instructions access memory using base-plus-offset addressing. The register fields are placed in the least-significant nibbles of the instruction word so that the load/store register and base register remain readable in hexadecimal dumps.

Bits	31–26	25–24	23	22–8	7–4	3–0
Field	**opcode**	**size**	**mode**	**offset15**	**rd/rs**	**base**

- **opcode** is the 6-bit major opcode. Load and store instructions use separate major opcodes.
- **size** encodes the width of the operation.

- **mode** flags extra load properties, determining if the operation is signed, unsigned or float, relative to the **size**.
- **offset15** is a signed 15-bit byte offset.
- **rd/rs** is the destination register field for loads.
- **rd/rs** is the source register field for stores.
- **base** is the base address register field.
- The effective address is computed as $base + \text{sext}(offset15)$.

Load Interpretation

For load instructions, **mode** selects whether sub-word loads are zero-extended for an unsigned load, sign-extended for a signed load; or if a word load is a binary32; or if a double word load's target is the upper or lower word;

Mnemonic	**size**	**mode**	Meaning	File
lbu	00	0	load unsigned byte	GPR
lb	00	1	load signed byte	GPR
lhu	01	0	load unsigned halfword	GPR
lh	01	1	load signed halfword	GPR
lw	10	0	load word	GPR
ld	10	1	load binary32	FPR
ldl	11	0	load binary64 lower	FPR
ldu	11	1	load binary64 upper	FPR

Store Interpretation

For store instruction, **mode** only applies to word-sized operations, determining if the target is a float, and which register.

Mnemonic	**size**	**mode**	Meaning	File
sb	00	0	store byte	GPR
sh	01	0	store halfword	GPR
sw	10	0	store word	GPR
sd	10	1	store binary32	FPR
sdl	11	0	store lower binary64	FPR
sdu	11	1	store upper binary64	FPR

Addressing Examples

The **rd/rs** registers belong to the file targeted by the instruction. The base registers are always from the GPR. Examples of M-type load operations include:

- **lb $rd, [$base, offset]**
- **lbu $rd, [$base, offset]**
- **lh $rd, [$base, offset]**
- **ldl #rd, [$base, offset]**
- **ld #rd, [$base, offset]**

Examples of M-type store operations include:

- `sb $rs, [$base, offset]`
- `sh $rs, [$base, offset]`
- `sdu #rs, [$base, offset]`

Floating-point Expansion

The floating-point instructions operate only on the floating-point registers. As the registers are 64-bit, binary32 loads expand their IEEE-754 binary32 value to binary64 representation, and binary32 stores convert the register's binary64 value to binary32.

Floating-point loads update the floating-point condition flags according to the resulting value. In particular, `N` and `Z` describe the numeric result, while `I`, `U`, `X`, and `E` indicate the corresponding floating-point result or conversion conditions. Integer-only flags `C` and `V` are cleared.

The binary64 half-load and half-store operations transfer raw 32-bit portions of a floating-point register without converting or otherwise interpreting the transferred portion. A half-load modifies only the targeted 32 bits of the destination floating-point register; the non-targeted portion is preserved.

After each half-load, the condition flags are updated from the complete 64-bit contents of the destination floating-point register as though those contents were an IEEE-754 binary64 value. Consequently, when constructing a binary64 value using two half-loads, the flag state produced by the first half-load is generally not meaningful. The flags should be considered to describe the loaded binary64 value only after both halves have been loaded.

The following stores and then loads a binary64 value without loss of precision:

```
sdu     #1, [$1, 0]
sdl     #1, [$1, 4]
ldu     #2, [$1, 0]
ldl     #2, [$1, 4]
```

Unlike the U-type instructions, these operations do **not** modify the non-targeted portion of the floating-point register. The order of the two half operations therefore does not affect the final register value, although it may affect the transient condition flags produced between the two loads.

These operation sequences also form the pseudo-instructions `sdd` and `ldd`. Note that the ordering here is deliberately big-endian to match the rest of the architecture.

4.4.7 BF-Type: Flag-Based Conditional Branches

BF-type instructions branch according to the current condition flags in the status register. They test a single existing status flag and do not perform arithmetic, comparison, or interpretation of the flag state.

BF-type instructions do not contain register fields, so the condition field and branch offset occupy the instruction payload.

Bits	31–26	25–22	21–0
Field	`opcode`	`cond`	`offset22`

- `opcode` is the 6-bit major opcode.
- `cond` is the 4-bit flag-condition field.
- `offset22` is a signed 22-bit PC-relative branch offset.

- Branch offsets are measured in 32-bit instruction words.
- The encoded offset is shifted left by two bits to form a byte offset.
- The branch target is relative to the address of the instruction following the branch.
- BF-type instructions test existing condition flags; they do not perform a new comparison or otherwise interpret the flags.
- BF-type instructions do not update condition flags.

The `cond` field identifies one of the eight architectural condition flags, together with whether the branch tests for that flag being set or clear. The available flags are `N`, `Z`, `C`, `V`, `E`, `I`, `U`, and `X`. Examples of BF-type operations include:

Instruction	Meaning
`bf.ns label`	Branch if `N` is set.
`bf.nc label`	Branch if `N` is clear.
`bf.zs label`	Branch if `Z` is set.
`bf.zc label`	Branch if `Z` is clear.
`bf.cs label`	Branch if `C` is set.
`bf.cc label`	Branch if `C` is clear.
`bf.vs label`	Branch if `V` is set.
`bf.vc label`	Branch if `V` is clear.
`bf.es label`	Branch if `E` is set.
`bf.ec label`	Branch if `E` is clear.
`bf.is label`	Branch if `I` is set.
`bf.ic label`	Branch if `I` is clear.
`bf.us label`	Branch if `U` is set.
`bf.uc label`	Branch if `U` is clear.
`bf.xs label`	Branch if `X` is set.
`bf.xc label`	Branch if `X` is clear.

BF-type branches have no signed, unsigned, integer, or floating-point comparison semantics. Comparisons requiring interpretation of register values are performed using the register-comparison branch instructions.

4.4.8 BC-Type: Register-Comparison Branches

BC-type instructions compare two registers and branch according to the specified comparison. The register fields are placed in the least-significant nibbles of the instruction word so that the compared registers remain readable in hexadecimal instruction dumps.

Bits	31–26	25–22	21–8	7–4	3–0
Field	`opcode`	`cond`	`offset14`	`ra`	`rb`

- `opcode` is the 6-bit major opcode. This will determine if the operation is GPR or FPR. Note that operation codes in the GPR instruction may not map to the same operation in the FPR instruction.
- `cond` is the 4-bit comparison condition-code field.
- `offset14` is a signed 14-bit PC-relative branch offset.
- `ra` is the first comparison register field.

- **rb** is the second comparison register field.
- Branch offsets are measured in 32-bit instruction words.
- The encoded offset is shifted left by two bits to form a byte offset.
- The branch target is relative to the address of the instruction following the branch.
- BC-type instructions compare **ra** and **rb** directly.
- BC-type instructions do not depend on existing condition flags.
- BC-type instructions do not update condition flags.

The low byte of a BC-type instruction contains the compared register fields. This makes the two comparison operands directly visible in the least-significant two hexadecimal digits of the instruction word.
Examples of BC-type operations include:

Instruction	Meaning
`b.eq $ra, $rb, label`	Branch to **label** if **ra** equals **rb**.
`b.ne $ra, $rb, label`	Branch to **label** if **ra** does not equal **rb**.
`bd.lt #ra, #rb, label`	Branch to **label** if **ra** is less than **rb**.
`b.ge $ra, $rb, label`	Branch to **label** if **ra** is greater than or equal to **rb**, treating both as signed values.
`b.ltu $ra, $rb, label`	Branch to **label** if **ra** is less than **rb**, treating both as unsigned values.
`bd.ge #ra, #rb, label`	Branch to **label** if **ra** is greater than or equal to **rb**.

4.4.9 J-Type: PC-Relative Jump and Call

J-type instructions perform unconditional PC-relative control transfer. J-type instructions do not contain register fields, so the full instruction payload is available for the PC-relative offset.

Bits	31–26	25–0
Field	**opcode**	**offset26**

- **opcode** is the 6-bit major opcode.
- **offset26** is a signed 26-bit PC-relative offset.
- Jump offsets are measured in 32-bit instruction words.
- The encoded offset is shifted left by two bits to form a byte offset.
- The jump target is relative to the address of the instruction following the jump or call instruction.
- J-type instructions do not update condition flags.

Examples of J-type operations include:

- `jmp label`
- `call label`

The **jmp** instruction changes the program counter to the computed target address. The **call** instruction also writes the return address to the architectural link register, **$15**.

4.4.10 JR-Type: Register-Indirect Jump and Call

JR-type instructions perform register-indirect control transfer. The register fields are placed in the least-significant nibbles of the instruction word so that the link register and target register remain readable in hexadecimal dumps.

Bits	31–26	25–8	7–4	3–0
Field	`opcode`	`func/reserved`	`rd/link`	`target`

- `opcode` is the 6-bit major opcode.
- `func/reserved` selects the register-indirect control-transfer operation or is reserved.
- `rd/link` is the destination link register field for register-indirect calls.
- `target` is the target address register field.
- For plain register jumps, `rd/link` must be encoded as `$0`.
- For register-indirect calls, `rd/link` receives the return address.
- The standard ABI link register is `$15`.
- Register-indirect targets must be 4-byte aligned.
- JR-type instructions do not update condition flags.

Examples of JR-type operations include:

- `jr $target`
- `jalr $rd, $target`
- `ret`

In VIV-32 v0.2, `ret` is defined as a pseudo-instruction for `jr $15`.

4.4.11 X-Type: System and Control Operations

X-type instructions access system facilities, control registers, traps, interrupt state, and processor execution state. The register fields are placed in the least-significant nibbles of the instruction word so that source and destination registers remain readable in hexadecimal dumps.

Bits	31–26	25–8	7–4	3–0
Field	`opcode`	`sysfunc`	`rd`	`rs`

- `opcode` is the 6-bit major opcode.
- `sysfunc` is an 18-bit field divided into `sysop` and `payload`.
- `sysop` selects the system or control operation.
- `payload` supplies operation-specific data, such as a trap immediate or control-register number.
- `rd` is the destination register field when required.
- `rs` is the source register field when required.

- Instructions that do not require **$rd** or **$rs** must encode unused register fields as **$0**.
- X-type instructions do not update condition flags unless explicitly specified by the instruction set chapter.

Examples of X-type operations include:

- **nop**
- **halt**
- **trap imm**
- **syscall**
- **iret**
- **ei**
- **di**
- **rdpc $rd**
- **mrs $rd, %cr**
- **msr %cr, $rs**

For control-register access instructions, the selected control register is encoded in the **creg4** field within the X-type **payload**. The **$rd** and **$rs** fields are used only for the general-purpose register operand.

The detailed interpretation of **sysfunc**, including **sysop**, **payload**, **trapimm12**, and **creg4**, is defined in the system and control encoding section.

4.5 Major Opcode Map

The 6-bit **opcode** field identifies the primary instruction class. Most instruction classes use additional fields, such as **func**, **mode**, **cond**, or **sysfunc**, to select the specific instruction within that class.

Opcode	Format	Class	Instructions
`0x00`	X	System/control	`nop`, `halt`, `trap`, `syscall`, `iret`, `ei`, `di`, `rdpc`, `mrs`, `msr`
`0x01`	R	Register ALU	`add`, `sub`, `and`, `or`, `xor`, `not`, `neg`, `cmp`, `addd`, `subd`, `andd`, `ord`, `xord`, `notd`, `negd`, `cmpd`
`0x02`	I	Immediate arithmetic/compare	`addi`, `subi`, `cmpi`
`0x03`	I	Immediate logical	`andi`, `ori`, `xori`
`0x04`	R	Register shifts	`shl`, `shr`, `sar`, `shld`, `shrd`, `sard`
`0x05`	I	Immediate shifts	`shli`, `shri`, `sari`
`0x06`	BI	Bit immediate	`shldi`, `shrdi`, `sardi`, `btst`, `btstd`, `bset`, `bsetd`, `bclr`, `bclrd`, `btgl`, `btgld`
`0x07`	R2	Multiply/divide	`mul`, `mulu`, `div`, `divu`, `muld`, `divd`
`0x08`	U	Constant construction	`lui`, `lli`, `lhi`
`0x09`	M	Load	`lb`, `lbu`, `lh`, `lhu`, `lw`, `ld`, `ldl`, `ldu`
`0x0A`	M	Store	`sb`, `sh`, `sw`, `sd`, `sdl`, `sdu`
`0x0B`	BF	Flag branch	`bf.*`
`0x0C`	BC	GPR branch	`b.*`
`0x0D`	J	PC-relative jump	`jmp`
`0x0E`	J	PC-relative call	`call`
`0x0F`	JR	Register jump/call	`jr`, `jalr`
`0x10`	R	Inter-file transfer	`disf`, `itod`, `utod`, `ftod`, `dtoi`, `dtou`, `dtof`
`0x11`	BC	FPR branch	`bd.*`
`0x12-0x3F`	–	Reserved	Reserved for future extensions

Opcode values not listed as assigned are reserved. Reserved opcode values must not be emitted by conforming assemblers. Executing an instruction with a reserved opcode raises an `IllegalInstructionException`.

4.6 Class-Specific Encoding Values

This section defines the class-specific function and mode fields used by the instruction classes listed in the major opcode map.

4.6.1 Register ALU Function Codes

Register ALU instructions use opcode `0x01` and the R-type instruction format.

file	func value	Mnemonic	Meaning
0x0	0x000	add	Add registers, GPR
0x1	0x000	addd	Add registers, FPR
0x0	0x001	sub	Sub registers, GPR
0x1	0x001	subd	Sub registers, FPR
0x0	0x002	and	Bitwise AND, GPR
0x1	0x002	andd	Bitwise AND, FPR
0x0	0x003	or	Bitwise OR, GPR
0x1	0x003	ord	Bitwise OR, FPR
0x0	0x004	xor	Bitwise XOR, GPR
0x1	0x004	xord	Bitwise XOR, FPR
0x0	0x005	not	Bitwise NOT, GPR
0x1	0x005	notd	Bitwise NOT, FPR
0x0	0x006	neg	Negate register, GPR
0x1	0x006	negd	Negate register, FPR
0x0	0x007	cmp	Compare registers, GPR
0x1	0x007	cmpd	Compare registers, FPR
0x0-0x3	0x008-0xFFF	reserved	Reserved

For **not**, **notd**, **neg**, **negd**, **cmp**, **cmpd** the **rb** field is reserved and must be encoded as **0** by conforming assemblers.

4.6.2 Immediate Arithmetic and Compare Mode Codes

Immediate arithmetic and compare instructions use opcode **0x02** and the I-type instruction format.

mode value	Mnemonic	Meaning
0x0	addi	Add sign-extended immediate
0x1	subi	Subtract sign-extended immediate
0x2	cmpi	Compare with sign-extended immediate
0x3	reserved	Reserved

For **cmpi**, the **rd** field is reserved and must be encoded as **r0** by conforming assemblers. The instruction updates condition flags but does not write a general-purpose register.

4.6.3 Immediate Logical Mode Codes

Immediate logical instructions use opcode **0x03** and the I-type instruction format.

mode value	Mnemonic	Meaning
0x0	andi	Bitwise AND with zero-extended immediate
0x1	ori	Bitwise OR with zero-extended immediate
0x2	xori	Bitwise XOR with zero-extended immediate
0x3	reserved	Reserved

4.6.4 Register Shift Function Codes

Register shift instructions use opcode **0x04** and the R-type instruction format.

file	func value	Mnemonic	Meaning
0x0	0x000	shl	Logical shift left by register count, GPR
0x1	0x000	shld	Logical shift left by register count, FPR
0x0	0x001	shr	Logical shift right by register count, GPR
0x1	0x001	shrd	Logical shift right by register count, FPR
0x0	0x002	sar	Arithmetic shift right by register count, GPR
0x1	0x002	sard	Arithmetic shift right by register count, FPR
0x0-0x3	0x003-0xFFF	reserved	Reserved

4.6.5 Immediate Shift Mode Codes

Immediate shift instructions use opcode 0x05 and the I-type instruction format.

mode value	Mnemonic	Meaning
0x0	shli	Logical shift left by immediate count
0x1	shri	Logical shift right by immediate count
0x2	sari	Arithmetic shift right by immediate count
0x3	reserved	Reserved

4.6.6 Bit Immediate Mode Codes

Bit immediate instructions use opcode 0x06 and the BI-type instruction format.

mode	file	Mnemonic	Meaning
0x0	0x0	btst	Test immediate bit index, GPR
0x0	0x1	btstd	Test immediate bit index, FPR
0x1	0x0	bset	Set immediate bit index, GPR
0x1	0x1	bsetd	Set immediate bit index, FPR
0x2	0x0	bclr	Clear immediate bit index, GPR
0x2	0x1	bclrd	Clear immediate bit index, FPR
0x3	0x0	btgl	Toggle immediate bit index, GPR
0x3	0x1	btgld	Toggle immediate bit index, FPR
0x4	0x1	shldi	Logical shift left, FPR
0x5	0x1	shrdi	Logical shift right, FPR
0x6	0x1	sardi	Arithmetic shift right, FPR

The value in imm14 is the bit index or shift count, according to the instruction.
For btst, and btstd the rd field is reserved and must be encoded as 0 by conforming assemblers.
The instruction updates condition flags but does not write to a register.

4.6.7 Multiply and Divide Function Codes

Multiply and divide instructions use opcode 0x07 and the R2-type instruction format.

file	func value	Mnemonic	Meaning
0	0x00	mul	Signed full-width multiply, GPR
1	0x00	muld	Floating-point multiply, FPR
0	0x01	mulu	Unsigned full-width multiply, GPR
1	0x01	-	Reserved
0	0x02	div	Signed divide with remainder, GPR
1	0x02	divd	Floating-point divide, FPR
0	0x03	divu	Unsigned divide with remainder, GPR
1	0x03	-	Reserved
-	0x04-0xFF	reserved	Reserved

For GPR multiply instructions, `rd0` receives the low 32 bits of the product and `rd1` receives the high 32 bits of the product. FPR multiply results are stored in a single register, with `rd1` being `0`.
For GPR divide instructions, `rd0` receives the quotient and `rd1` receives the remainder. FPR divide results are stored in a single register, with `rd1` being `0`.

4.6.8 Constant Construction Mode Codes

Constant construction instructions use opcode `0x08` and the U-type instruction format.

`mode` value	Mnemonic	Meaning
`0x00`	`lui`	Load upper immediate, clear lower half
`0x01`	`lli`	Load lower immediate, clear upper half
`0x02`	`lhi`	Load upper immediate, preserve lower half
`0x03-0x3F`	reserved	Reserved

For `lui`, the `imm16` field is placed in bits 31–16 of the destination register and bits 15–0 are cleared.
For `lli`, the `imm16` field is placed in bits 15–0 of the destination register and bits 31–16 are cleared.
For `lhi`, the `imm16` field is placed in bits 31–16 of the destination register, without modifying bits 15–0.

4.7 Memory Encoding Values

Memory instructions use the M-type instruction format. Load instructions use opcode `0x09`. Store instructions use opcode `0x0A`.

4.7.1 Load Encoding

Load instructions use opcode `0x09`. The `mode` field selects whether sub-word loads are sign-extended or zero-extended; whether word loads are integer or binary32; whether double loads are upper or lower.

Mnemonic	`size`	`mode`	Meaning
`lbu`	`00`	`0`	Load byte and zero-extend to 32 bits
`lb`	`00`	`1`	Load byte and sign-extend to 32 bits
`lhu`	`01`	`0`	Load halfword and zero-extend to 32 bits
`lh`	`01`	`1`	Load halfword and sign-extend to 32 bits
`lw`	`10`	`0`	Load word
`ld`	`10`	`1`	Load binary32
`ldl`	`11`	`0`	Load lower binary64
`ldu`	`11`	`1`	Load upper binary64

For byte and halfword loads, the loaded value is extended to 32 bits according to the `mode` field. `ld` extends upon loading from binary32 in memory to binary64 in register, and loads into a floating-point register. `ldl` and `ldu` load unchanged into their respective floating-point register.

4.7.2 Store Encoding

Store instructions use opcode `0x0A`.

Mnemonic	size	mode	Meaning
sb	00	0	Store byte
-	00	1	Reserved
sh	01	0	Store halfword
-	01	1	Reserved
sw	10	0	Store word
sd	10	1	Store binary32
sdl	11	0	Store lower binary64
sdu	11	1	Store upper binary64

Executing an invalid store instruction raises an `IllegalInstructionException`.

4.7.3 Memory Access Size Field

The `size` field has the same basic interpretation for load and store instructions.

size value	Access size	Alignment requirement
00	Byte	Any address
01	Halfword	2-byte aligned address
10	Word	4-byte aligned address
11	Double-half	4-byte aligned address

Misaligned halfword, word and double-half memory accesses raise a `MisalignedAccessException`. Byte accesses are always naturally aligned.

4.8 Inter-File Transfer

Inter-file transfer instructions convert values between the general-purpose and floating-point register files. They use the R-type instruction format and major opcode `0x10`.

The interpretation of the source and destination operands is determined by the transfer function. Integer conversions interpret a general-purpose register as a signed or unsigned 32-bit integer. Floating-point format conversions interpret a general-purpose register as an IEEE-754 binary32 bit pattern or a floating-point register as an IEEE-754 binary64 value.

file	func value	Mnemonic	Meaning
0x0	0x000	disf	Test whether a binary64 value is representable as binary32
0x1	0x001	utod	Unsigned 32-bit integer to binary64
0x1	0x002	itod	Signed 32-bit integer to binary64
0x1	0x003	ftod	Binary32 to binary64
0x0	0x001	dtou	Binary64 to unsigned 32-bit integer
0x0	0x002	dtoi	Binary64 to signed 32-bit integer
0x0	0x003	dtof	Binary64 to binary32
-	0x004-0xFFF	reserved	Reserved

For conversion instructions with a destination register, `rd` identifies the destination register and `ra` identifies the source register. The register file associated with each operand is determined by the transfer function. Unless otherwise specified, `rb` is unused and must be encoded as zero.

The `utod` and `itod` instructions perform numeric integer-to- floating-point conversion. The `dtou` and `dtoi` instructions perform numeric floating-point-to-integer conversion.

The `ftod` instruction interprets the 32 bits held in its general-purpose source register as an IEEE-754 binary32 value and converts that value to binary64. The `dtof` instruction converts an

IEEE-754 binary64 value to binary32 and writes the resulting 32-bit IEEE-754 representation to its general-purpose destination register.

Floating-point conversion instructions update the applicable **NZEIUX** flags according to the conversion result. The integer-only **C** and **V** flags are cleared unless otherwise specified.

The **disf** instruction performs a representability test and does not transfer a value between register files. Its precise flag behaviour is defined with the instruction semantics.

4.9 Branch Condition Codes

Branch instructions use a 4-bit **cond** field to select the branch condition. The interpretation of this field depends on the branch instruction type.

BF-type instructions interpret **cond** as a direct test of one of the architectural status flags. BC-type instructions interpret **cond** as a comparison condition applied directly to their register operands.

The two instruction types therefore use independent condition-code assignments.

4.9.1 Flag-Based Branch Conditions

BF-type instructions use opcode **0x0B**. The **cond** field selects one of the eight architectural condition flags and whether the branch tests that flag for being set or clear.

BF-type instructions do not interpret the flags as the result of a signed, unsigned, integer, or floating-point comparison. They simply test the current flag state.

cond value	Suffix	Branch is taken when	Meaning
0x0	**ns**	**N** is set	Negative set
0x1	**nc**	**N** is clear	Negative clear
0x2	**zs**	**Z** is set	Zero set
0x3	**zc**	**Z** is clear	Zero clear
0x4	**cs**	**C** is set	Carry set
0x5	**cc**	**C** is clear	Carry clear
0x6	**vs**	**V** is set	Overflow set
0x7	**vc**	**V** is clear	Overflow clear
0x8	**es**	**E** is set	Arithmetic error set
0x9	**ec**	**E** is clear	Arithmetic error clear
0xA	**is**	**I** is set	Infinity set
0xB	**ic**	**I** is clear	Infinity clear
0xC	**us**	**U** is set	Underflow set
0xD	**uc**	**U** is clear	Underflow clear
0xE	**xs**	**X** is set	Inexact set
0xF	**xc**	**X** is clear	Inexact clear

The condition-code suffix is appended to the **bf** mnemonic. For example, **bf.zs label** branches when the zero flag is set, while **bf.ec label** branches when the arithmetic-error flag is clear. BF-type instructions test only the existing processor-status flags. They do not perform a comparison and do not update the status register.

4.9.2 Register-Comparison Branch Conditions

BC-type instructions use opcode **0x0C** for the GPR, and **0x11** for the FPR. The **cond** field selects a direct comparison between **ra** and **rb**.

Suffix	Branch is taken when	Comparison type
`eq`	`ra == rb`	Equality
`ne`	`ra != rb`	Inequality
`lt`	`ra < rb`	Signed
`le`	`ra <= rb`	Signed
`gt`	`ra > rb`	Signed
`ge`	`ra >= rb`	Signed
`ltu`	`ra < rb`	Unsigned
`leu`	`ra <= rb`	Unsigned
`gtu`	`ra > rb`	Unsigned
`geu`	`ra >= rb`	Unsigned

BC-type instructions do not read or update the condition flags. Condition-code values `0xA`–`0xF` are reserved for BC-type instructions.

4.10 Jump and Call Encoding Values

4.10.1 PC-Relative Jump and Call Opcodes

PC-relative jump and call instructions use the J-type instruction format.

Opcode	Mnemonic	Meaning
`0x0D`	`jmp`	PC-relative unconditional jump
`0x0E`	`call`	PC-relative call using `$15` as the link register

Both instructions use the signed `offset26` field. The encoded offset is measured in 32-bit instruction words and is shifted left by two bits to form the byte offset. The target address is relative to the address of the instruction following the jump or call.
For `call`, the return address is written to the architectural link register, `$15`.

4.10.2 Register-Indirect Jump and Call Function Codes

Register-indirect jump and call instructions use opcode `0x0F` and the JR-type instruction format.

`func` value	Mnemonic	Meaning
`0x00000`	`jr`	Jump to address in `target`
`0x00001`	`jalr`	Jump to address in `target` and write link register
`0x00002-0x3FFFF`	reserved	Reserved

For `jr`, the `rd/link` field is reserved and must be encoded as `0` by conforming assemblers.
For `jalr`, the `rd/link` field selects the destination link register. The standard ABI link register is `$15`.
Register-indirect targets must be 4-byte aligned. A misaligned register-indirect target raises a `MisalignedFetchException`.

4.11 System and Control Encoding Values

System and control instructions use opcode `0x00` and the X-type instruction format. The 18-bit `sysfunc` field is divided into a system operation field and a payload field.

Bits within `sysfunc`	Instruction bits	Field	Meaning
17–12	25–20	`sysop`	System operation selector
11–0	19–8	`payload`	Operation-specific payload

The interpretation of `payload` depends on the selected `sysop`. For instructions that do not use a payload, the `payload` field is reserved and must be encoded as zero by conforming assemblers.

4.11.1 System Operation Codes

`sysop` value	Mnemonic	Meaning
`0x00`	`nop`	No operation
`0x01`	`halt`	Halt processor execution
`0x02`	`trap`	Raise software trap
`0x03`	`syscall`	Raise system-call exception
`0x04`	`iret`	Return from interrupt or exception
`0x05`	`ei`	Enable maskable interrupts
`0x06`	`di`	Disable maskable interrupts
`0x07`	`rdpc`	Read program counter
`0x08`	`mrs`	Move from control register
`0x09`	`msr`	Move to control register
`0x0A-0x3F`	reserved	Reserved

4.11.2 System Payload Encoding

Mnemonic	`sysop`	`payload` use	Payload requirement
`nop`	`0x00`	Unused	Must be zero
`halt`	`0x01`	Unused	Must be zero
`trap`	`0x02`	`trapimm12`	Software-defined trap immediate
`syscall`	`0x03`	Unused	Must be zero
`iret`	`0x04`	Unused	Must be zero
`ei`	`0x05`	Unused	Must be zero
`di`	`0x06`	Unused	Must be zero
`rdpc`	`0x07`	Unused	Must be zero
`mrs`	`0x08`	`creg4`	Bits 11–4 must be zero
`msr`	`0x09`	`creg4`	Bits 11–4 must be zero

For `trap`, the 12-bit `payload` field is interpreted as an unsigned software trap immediate, `trapimm12`. This data is stored in `%edata`.
For `mrs` and `msr`, the low 4 bits of `payload` encode the selected control register. The upper 8 bits of `payload` are reserved and must be encoded as zero by conforming assemblers.

4.11.3 System Instruction Register Fields

Mnemonic	`rd`	`rs`	Register-field use
`nop`	`r0`	`r0`	Unused
`halt`	`r0`	`r0`	Unused
`trap`	`r0`	`r0`	Unused
`syscall`	`r0`	`r0`	Unused
`iret`	`r0`	`r0`	Unused
`ei`	`r0`	`r0`	Unused
`di`	`r0`	`r0`	Unused
`rdpc`	destination	`r0`	`rd` receives the program counter value
`mrs`	destination	`r0`	`rd` receives the selected control-register value
`msr`	`r0`	source	`rs` supplies the value written to the selected control register

For instructions that do not use `rd` or `rs`, unused register fields must be encoded as `r0` by conforming assemblers.

4.11.4 Control Register Numbers

Control-register access instructions use the low 4 bits of the X-type **payload** field to select a control register.

creg4 value	Name	Meaning
0x0	%pc	Program counter, read-only
0x1	%sr	Status register
0x2	%epc	Exception program counter
0x3	%esr	Exception status register
0x4	%ecause	Exception cause
0x5	%edata	Exception-specific data
0x6	%evbase	Exception vector base address
0x7-0xF	reserved	Reserved

Executing **mrs** or **msr** with a reserved **creg4** value raises an **IllegalInstructionException**.

4.12 NOP Encoding

VIV-32 defines the all-zero instruction word as **nop**.

Field	Value	Meaning
opcode	0x00	X-type system/control instruction
sysop	0x00	nop
payload	0x000	Unused
rd	r0	Unused
rs	r0	Unused

The complete encoding of **nop** is therefore:

```
0x00000000
```

This guarantee allows zero-filled memory to decode as a valid no-operation instruction. It also provides a safe default encoding for padding and alignment.

4.13 Flag Update Policy

Instruction encodings do not contain a general flag-update bit in VIV-32 v0.2. Whether an instruction updates condition flags is defined by the instruction semantics in Chapter 5.

Arithmetic, logical, comparison, shift, bit-test, multiply, divide and load instructions update flags as specified by their instruction definitions. Store, branch, jump, call, and most system/control instructions do not update flags unless explicitly stated.

4.14 Reserved Encodings

Reserved opcode values, function codes, mode codes, condition codes, size values, system operation codes, payload bits, control-register numbers, and register-field combinations must not be emitted by conforming assemblers.

Unless otherwise specified, executing an instruction with a reserved encoding raises an illegal-instruction exception.

Chapter 5

Instruction Set

This chapter defines the architectural meaning of each instruction. Opcode assignments and binary layouts are defined separately by the instruction encoding and opcode map.

5.1 Notation

5.1.1 Instruction Notation

- `$`: prefix for general-purpose registers.
- `#`: prefix for floating-point registers.
- `%`: prefix for named control registers.
- `rd`: destination register.
- `rd0`: first destination register.
- `rd1`: second destination register.
- `ra`: first source register.
- `rb`: second source register.
- `rs`: source register.
- `immN`: immediate N-bit value encoded in the instruction.
- `offsetN`: signed N-bit `%pc`- or base-relative offset.
- `x.y`: bit `y` of register `x`.
- `%pc`: program counter.
- `%sr`: processor status register.
- `N`: negative flag.
- `Z`: zero flag.
- `C`: carry flag.
- `V`: overflow flag.
- `I`: infinity flag.

- `U`: underflow flag.
- `X`: inexact flag.
- `E`: arithmetic error flag.

5.1.2 Assignment Notation

- The notation `$rd` ← `x` means that value `x` is written to register `$rd`.
- The notation `%pc` ← `x` means that execution continues from address `x`.
- The notation `[x]` means memory at address `x`.
- The notation `SEXT(x)` means sign-extension of `x` to 32 bits.
- The notation `ZEXT(x)` means zero-extension of `x` to 32 bits.
- The notation `a:b` means a register pair, with `a` holding the upper 32 bits and `b` holding the lower 32 bits unless otherwise specified.

5.1.3 Immediate Values

- Unless otherwise specified, immediate operands are sign-extended to 32 bits before use.
- Arithmetic immediate instructions sign-extend their immediate operands.
- Logical immediate instructions zero-extend their immediate operands.
- Shift counts are interpreted as unsigned values.
- Bit indexes are interpreted as unsigned values in the range 0–31 for general purpose registers, and 0–63 for floating-point registers.

5.1.4 Conditional Suffixes

- Conditional instruction mnemonics use a dot-separated condition suffix.
- Register-comparison branches use the form `b[d].<cond>`, where `d` signifies the operands are floating-point registers.
- Flag-based branches use the form `bf.<cond>`.
- Future conditional instructions should use the same suffix convention.

Suffix	Meaning	Basis
`.eq`	equal	register comparison
`.ne`	not equal	register comparison
`.lt`	signed less than	signed register comparison
`.le`	signed less than or equal	signed register comparison
`.gt`	signed greater than	signed register comparison
`.ge`	signed greater than or equal	signed register comparison
`.ltu`	unsigned less than	unsigned register comparison
`.leu`	unsigned less than or equal	unsigned register comparison
`.gtu`	unsigned greater than	unsigned register comparison
`.geu`	unsigned greater than or equal	unsigned register comparison

Flag-based branches use a separate set of suffixes:

Suffix	Meaning	Basis
`.ns`	negative set	`N = 1`
`.nc`	negative clear	`N = 0`
`.zs`	zero set	`Z = 1`
`.zc`	zero clear	`Z = 0`
`.cs`	carry set	`C = 1`
`.cc`	carry clear	`C = 0`
`.vs`	overflow set	`V = 1`
`.vc`	overflow clear	`V = 0`
`.es`	arithmetic error set	`E = 1`
`.ec`	arithmetic error clear	`E = 0`
`.is`	infinity set	`I = 1`
`.ic`	infinity clear	`I = 0`
`.us`	underflow set	`U = 1`
`.uc`	underflow clear	`U = 0`
`.xs`	inexact set	`X = 1`
`.xc`	inexact clear	`X = 0`

5.2 Flag Behaviour

Condition flags describe the result or status of the most recent flag-setting operation.

- Arithmetic and logical instructions update condition flags by default.
- Comparison instructions update condition flags without storing a result.
- Load instructions update condition flags according to the resulting register value.
- Store instructions do not update condition flags.
- Branch, jump, call, and return instructions do not update condition flags.
- System instructions update condition flags only when explicitly specified.

Unless otherwise specified by an instruction, all condition flags are updated or cleared according to the rules in this section.

5.2.1 Negative Flag

The negative flag `N` indicates that the numeric result is negative.

- For integer results, `N` is set when bit 31 of the result is set.
- For floating-point results, `N` is set when the numeric binary64 result is negative.
- Floating-point `+0.0` and `-0.0` do not set `N`.

5.2.2 Zero Flag

The zero flag `Z` indicates that the numeric result is zero.

- For integer and logical results, `Z` is set when all result bits are zero.
- For floating-point results, `Z` is set when the result is either `+0.0` or `-0.0`.
- Otherwise, `Z` is cleared.

5.2.3 Carry Flag

The carry flag `C` is used by integer arithmetic.

- For integer addition, `C` is set when an unsigned carry is produced from bit 31.
- For integer subtraction, `C` is set when no unsigned borrow is required and cleared when a borrow is required.
- For successful integer division, `C` is set when the remainder is non-zero.
- Floating-point operations clear `C` unless otherwise specified.
- Logical operations clear `C` unless otherwise specified.

5.2.4 Overflow Flag

The overflow flag `V` is used by integer arithmetic.

- For signed integer arithmetic, `V` is set when the mathematical result cannot be represented as a signed 32-bit two's-complement integer.
- Floating-point operations clear `V` unless otherwise specified.
- Logical operations clear `V` unless otherwise specified.

5.2.5 Arithmetic Error Flag

The arithmetic error flag `E` indicates an exceptional or invalid arithmetic or conversion condition that does not trap in VIV-32 v0.2.

- Integer divide-by-zero sets `E`.
- Integer division that cannot produce a representable quotient sets `E`.
- Floating-point divide-by-zero sets `E`.
- A floating-point operation producing `NaN` sets `E`.
- An unordered floating-point comparison involving `NaN` sets `E`.
- A floating-point to integer conversion of `NaN`, infinity, or a value outside the destination integer range sets `E`.
- Arithmetic and conversion operations producing an ordinary valid result clear `E`, unless otherwise specified.

5.2.6 Infinity Flag

The infinity flag `I` indicates that a floating-point result is infinite.

- `I` is set when the resulting floating-point value is positive or negative infinity.
- The presence of infinity does not by itself imply an arithmetic error; `E` is determined by the operation that produced or processed the value.
- Integer operations clear `I` unless otherwise specified.

5.2.7 Underflow Flag

The underflow flag `U` indicates that a floating-point result is too small in magnitude to be represented normally in the destination floating-point format.

- `U` is set when a floating-point arithmetic or conversion result enters the underflow range of its destination format.
- For binary64 arithmetic, the destination format is binary64.
- For binary64-to-binary32 conversion, the destination format is binary32.
- Integer operations clear `U` unless otherwise specified.

5.2.8 Inexact Flag

The inexact flag `X` indicates that a numeric result required rounding.

- `X` is set when the mathematically exact floating-point or conversion result cannot be represented exactly in the destination format.
- `X` may be set independently of `U`, `I`, or `E`.
- Integer operations clear `X` unless otherwise specified.

5.3 Arithmetic Instructions

Arithmetic operations can be performed on both general purpose and floating-point registers. General purpose, single-target-register operations wrap modulo 2^{32}.

5.3.1 `add, addd`

- Syntax: `add $rd, $ra, $rb` | `addd #rd, #ra, #rb`
- Operation: `rd` ← `ra + rb`

5.3.2 `addi`

- Syntax: `addi $rd, $ra, imm16`
- Operation: `rd` ← `ra + SEXT(imm16)`

5.3.3 `sub, subd`

- Syntax: `sub $rd, $ra, $rb | subd #rd, #ra, #rb`
- Operation: `rd` ← `ra - rb`

5.3.4 `subi`

- Syntax: `subi $rd, $ra, $imm16`
- Operation: `rd` ← `ra - SEXT(imm16)`

5.3.5 `neg, negd`

- Syntax: `neg $rd, $ra | negd #rd, #ra`
- Operation: `rd` ← `0 - ra`

5.4 Logical Instructions

Logical instructions operate on the raw bit representation of their operands. This behaviour is the same for general-purpose and floating-point registers; floating-point register contents are not interpreted as IEEE-754 values for the purposes of logical operations or flag generation.

For a logical operation:

- `N` is set from the most-significant bit of the result: bit 31 for a general-purpose register and bit 63 for a floating-point register.
- `Z` is set if all bits of the result are zero.
- `C` and `V` are cleared.
- `E` is cleared.
- `I`, `U`, and `X` are cleared.

5.4.1 `and, andd`

- Syntax: `and $rd, $ra, $rb | andd #rd, #ra, #rb`
- Operation: `rd ← ra & rb`

5.4.2 `andi`

- Syntax: `andi $rd, $ra, imm16`
- Operation: `rd ← ra & ZEXT(imm16)`

5.4.3 `or, ord`

- Syntax: `or $rd, $ra, $rb | ord #rd, #ra, #rb`
- Operation: `rd ← ra | rb`

5.4.4 `ori`

- Syntax: `ori $rd, $ra, imm16`
- Operation: `rd ← ra | ZEXT(imm16)`

5.4.5 `xor, xord`

- Syntax: `xor $rd, $ra, $rb | xord #rd, #ra, #rb`
- Operation: `rd ← ra ^ rb`

5.4.6 `xori`

- Syntax: `xori $rd, $ra, imm16`
- Operation: `rd ← ra ^ ZEXT(imm16)`

5.4.7 `not, notd`

- Syntax: `not $rd, $ra | notd #rd, #ra`
- Operation: `rd ← ~ ra`

5.5 Comparison Instructions

Floating-point comparisons use IEEE-754 binary64 numerical comparison semantics.

If either floating-point operand is `NaN`, the comparison is unordered. For a flag-setting floating-point comparison, an unordered comparison sets `E` and clears the ordered comparison flags. No ordering relationship is considered true for an unordered comparison.

For direct floating-point register-comparison branches, unordered comparison behaviour is defined by the branch condition itself: `bd.eq`, `bd.lt`, `bd.le`, `bd.gt`, and `bd.ge` do not branch, while `bd.ne` branches.

Floating-point comparison does not compare IEEE-754 encodings bitwise; values are compared numerically. In particular, `+0.0` and `-0.0` compare equal.

5.5.1 `cmp, cmpd`

- Syntax: `cmp $ra, $rb | cmpd #ra, #rb`
- Operation: compares `ra` with `rb` and updates the condition flags without storing a result.
- `cmp` performs a 32-bit integer comparison. The comparison updates the integer condition flags as though `ra - rb` had been evaluated, but not arithmetic result is written to a register.
- `cmpd` performs an IEEE 754 binary64 numerical comparison. `Z` is set when the operands compare equal and `N` is set when `ra` compares less than `rb`. `C` and `V` are cleared.
- If either operand of `cmpd` is `NaN`, the comparison is unordered. `E` is set and the ordered comparison flags are cleared.
- Only `%sr` is modified.

5.5.2 `cmpi`

- Syntax: `cmpi $ra, imm16`
- Operation: computes `ra - SEXT(imm16)` for flag results only.
- Only `%sr` is modified.

5.5.3 btst, btstd

- Syntax: `btst $ra, imm14 | btstd #ra, imm14`
- Operation: test bit `imm14` of the raw bit representation of `ra`.
- For `btst`, valid bit indices are `0-31`.
- For `btstd`, valid bit indices are `0-63`.
- On a successful bit test:
  - Z is set if the selected bit is clear, and cleared if the selected bit is set.
  - N, C, V, E, I, U, and X are cleared.
- If `imm14` is outside the valid bit-index range:
  - E is set.
  - N, Z, C, V, I, U, and X are cleared.
- No general-purpose or floating-point register is modified.

5.6 Shift Instructions

Shift instructions that use a register to hold the shift value always use a general purpose register. These instructions do not take the underlying format into account, so while their use on floating-point registers is nonsensical from a floating-point perspective, they open up the possibility of 64-bit integer manipulations for those who need such functionality.

5.6.1 Shift Register Behaviour

Shift instructions operate on the raw bit representation of the source register. This behaviour is the same for general-purpose and floating-point registers; floating-point register contents are not interpreted as IEEE-754 values for the purposes of shift operations or flag generation.

For a successful shift operation:

- `N` is set from the most-significant bit of the result: bit 31 for a general-purpose register and bit 63 for a floating-point register.
- `Z` is set if all bits of the result are zero.
- `C` contains the last bit shifted out of the register.
- `V` and `E` are cleared.
- `I`, `U`, and `X` are cleared.

If the shift count is greater than or equal to the bit-width of the target register:

- `E` is set.
- `N`, `Z`, `C`, `V`, `I`, `U`, and `X` are cleared.
- The destination register is left unchanged.

5.6.2 `shl, shld`

- Syntax: `shl $rd, $ra, $rb | shld #rd, #ra, $rb`
- Operation: `rd` ← `ra << rb`
- Vacated low bits are filled with zero.

5.6.3 `shli, shldi`

- Syntax: `shli $rd, $ra, imm14 | shldi #rd, #ra, imm14`
- Operation: `rd` ← `ra << imm14`
- Vacated low bits are filled with zero.

5.6.4 `shr, shrd`

- Syntax: `shr $rd, $ra, $rb | shrd #rd, #ra, $rb`
- Operation: logical right shift, `rd` ← `ra >> rb`
- Vacated high bits are filled with zero.

5.6.5 `shri, shrdi`

- Syntax: **`shri $rd, $ra, imm14 | shrdi #rd, #ra, imm14`**
- Operation: logical right shift, **`rd`** ← **`ra >> imm14`**
- Vacated high bits are filled with zero.

5.6.6 `sar, sard`

- Syntax: **`sar $rd, $ra, $rb | sard #rd, #ra, $rb`**
- Operation: arithmetic right shift, **`rd`** ← **`ra >> rb`**
- Vacated high bits are filled with the original sign bit of (ra).

5.6.7 `sari, sardi`

- Syntax: **`sari $rd, $ra, imm14 | sardi #rd, #ra, imm14`**
- Operation: arithmetic right shift, **`rd`** ← **`ra >> imm14`**
- Vacated high bits are filled with the original sign bit of (ra).

5.7 Bit Manipulation Instructions

For a successful bit-manipulation operation:

- N is set from the most-significant bit of the result: bit 31 for a general-purpose register and bit 63 for a floating-point register.
- Z is set if the resulting register value is zero.
- C, V, E, I, U, and X are cleared.

If the selected bit index is outside the valid range of the target register:

- E is set.
- N, Z, C, V, I, U, and X are cleared.
- The destination register is left unchanged.

5.7.1 `bset, bsetd`

- Syntax: **`bset $rd, $ra, imm14 | bsetd #rd, #ra, imm14`**
- Operation: sets **`ra.imm14`**, then writes the result to **`rd`**.

5.7.2 `bclr, bclrd`

- Syntax: **`bclr $rd, $ra, imm14 | bclrd #rd, #ra, imm14`**
- Operation: clears **`ra.imm14`**, then writes the result to **`rd`**.

5.7.3 `btgl, btgld`

- Syntax: **`btgl $rd, $ra, imm14 | btgld #rd, #ra, imm14`**
- Operation: toggles bit **`ra.imm14`** of **`ra`**, then writes the result to **`rd`**.

5.8 Multiply and Divide Instructions

Multiply and divide instructions on general purpose registers utilise two target registers, to hold upper and lower result and quotient and remainder respectively. Multiply and divide instructions on floating-point registers only require a single target register.

5.8.1 `mul`

- Syntax: **`mul $rd0, $rd1, $ra, $rb`**
- Operation: signed 32-bit multiply, **`rd1:rd0`** ← **`ra`** × **`rb`**
- The operands **`ra`** and **`rb`** are interpreted as signed 32-bit two's complement integers.
- The full signed 64-bit product is computed.
- The low 32 bits of the product are written to **`rd0`**.
- The high 32 bits of the product are written to **`rd1`**.
- On successful multiplication:
  - **`N`** is set from bit 31 of **`rd1`**.
  - **`Z`** is set if the full 64-bit product is zero.
  - Flags **`C`**, **`V`** and **`E`** are cleared.

5.8.2 `muld`

- Syntax: **`muld #rd, #ra, #rb`**
- Operation: IEEE-754 binary64 multiplication, **`rd`** ← **`ra`** × **`rb`**.
- Updates flags:
  - **`N`** is set when the numeric result is negative. `+0.0` and `-0.0` do not set **`N`**.
  - **`Z`** is set if the result is `+0.0` or `-0.0`.
  - **`C`** and **`V`** are cleared.
  - **`I`** is set if the result is positive or negative **`infinity`**.
  - **`U`** is set if the result underflows binary64 representation.
  - **`X`** is set if the exact mathematical result cannot be represented exactly in binary64 and rounding is required.
  - **`E`** is set if the operation produces an invalid floating-point result, including **`NaN`**; otherwise it is cleared.

5.8.3 `mulu`

- Syntax: **`mulu $rd0, $rd1, $ra, $rb`**
- Operation: unsigned 32-bit multiply, **`rd1:rd0`** ← **`ra`** × **`rb`**
- The operands **`ra`** and **`rb`** are interpreted as unsigned 32-bit integers.
- The full unsigned 64-bit product is computed.
- The low 32 bits of the product are written to **`rd0`**.

- The high 32 bits of the product are written to **rd1**.
- On successful multiplication:
  - **N** is set from bit 31 of **rd1**.
  - **Z** is set if the full 64-bit product is zero.
  - Flags **C** **V** and **E** are cleared.

5.8.4 div

- Syntax: **div $rd0, $rd1, $ra, $rb**
- Operation: signed 32-bit division.
- The operands **ra** and **rb** are interpreted as signed 32-bit two's-complement integers.
- On successful division, the signed quotient is written to **rd0** and the signed remainder is written to **rd1**.
- If **rb** is zero, division cannot be performed:
  - **E** is set.
  - **N**, **Z**, **C**, and **V** are cleared.
  - **rd0** and **rd1** are left unchanged.
- If **ra** is **0x80000000** and **rb** is **0xFFFFFFFF**, the mathematical quotient cannot be represented as a signed 32-bit integer:
  - **V** and **E** are set.
  - **N**, **Z**, and **C** are cleared.
  - **rd0** and **rd1** are left unchanged.
- Division errors do not trap in VIV-32 v0.2.
- On successful division:
  - **N** is set if the quotient is negative.
  - **Z** is set if the quotient is zero.
  - **C** is set if the remainder is non-zero.
  - **V** and **E** are cleared.
- As an integer operation, **I**, **U**, and **X** are cleared.

5.8.5 divd

- Syntax: **divd #rd, #ra, #rb**
- Operation: IEEE-754 binary64 division, **rd** ← **ra / rb**.
- Updates flags:
  - **N** is set when the numeric result is negative. **+0.0** and **-0.0** do not set **N**.
  - **Z** is set if the result is **+0.0** or **-0.0**.
  - **C** and **V** are cleared.
  - **I** is set if the result is positive or negative **infinity**.

- `U` is set if the result underflows binary64 representation.
- `X` is set if the exact mathematical result cannot be represented exactly in binary64 and rounding is required.
- `E` is set if the operation encounters an invalid or exceptional arithmetic condition, including division by zero or production of `NaN`; otherwise it is cleared.

5.8.6 `divu`

- Syntax: `divu $rd0, $rd1, $ra, $rb`
- Operation: unsigned 32-bit division.
- The operands `ra` and `rb` are interpreted as unsigned 32-bit integers.
- On successful division, the unsigned quotient is written to `rd0` and the unsigned remainder is written to `rd1`.
- If `rb` is zero, division cannot be performed:
  - `E` is set.
  - `N`, `Z`, `C`, and `V` are cleared.
  - `rd0` and `rd1` are left unchanged.
- Divide-by-zero does not trap in VIV-32 v0.2.
- On successful division:
  - `N` is set from bit 31 of the quotient.
  - `Z` is set if the quotient is zero.
  - `C` is set if the remainder is non-zero.
  - `V` and `E` are cleared.
- As an integer operation, `I`, `U`, and `X` are cleared.

5.9 Load and Store Instructions

Load and store instructions transfer data between registers and memory. VIV-32 uses base-plus-offset addressing for ordinary memory access. Multi-byte load and store operations are big-endian.

5.9.1 Address Calculation

The assembly syntax for memory addressing is:

```
reg, [$base, offset15]
```

In this syntax, `base` is a general-purpose register containing a 32-bit base address, and `offset15` is sign-extended to 32-bits. `reg` may be either a general purpose or floating-point register, depending on the encoded instruction.
The effective address is computed as:

```
address ← [base + SEXT(offset15)]
```

The computed address is interpreted according to the access size of the instruction. Byte accesses may occur at any address. Halfword, word and double-half accesses are subject to the alignment rules defined below.

5.9.2 Alignment

- Byte loads and stores may access any address.
- Halfword loads and stores must access a 2-byte aligned address.
- Word and double-half loads and stores must access a 4-byte aligned address.
- Misaligned halfword, word and double-half accesses raise a **`MisalignedAccessException`**:
  - **`%ecause`** receives the **`MisalignedAccessException`**.
  - The exception entry sequence is defined in the exceptions and interrupts chapter.

5.9.3 Flag Behaviour

- Load instructions update condition flags according to the resulting destination-register value.
- Integer loads update the integer result flags:
  - `N` reflects the sign of the loaded 32-bit result.
  - `Z` is set when the loaded result is zero.
  - `C`, `V`, `E`, `I`, `U`, and `X` are cleared.
- Floating-point loads update the floating-point result flags:
  - `N` is set when the resulting floating-point value is negative.
  - `Z` is set when the result is `+0.0` or `-0.0`.
  - `I` is set when the resulting value is positive or negative infinity.
  - `U`, `X`, and `E` are updated according to any conversion performed by the load.
  - `C` and `V` are cleared.
- The binary64 half-load instructions **`ldl`** and **`ldu`** update the flags from the complete 64-bit contents of the destination floating-point register after the selected half has been written.
- When a binary64 value is assembled using two half-loads, the flag state produced by the first half-load is generally not meaningful. The flags describe the complete loaded value only after both halves have been loaded.
- Store instructions do not update condition flags.

5.9.4 `lb`

- Syntax: **`lb $rd, [$base, offset]`**
- Operation: load signed byte.
- One byte is read from memory, sign-extended to 32 bits, then put in general purpose register **`$rd`**.

5.9.5 `lbu`

- Syntax: **`lbu $rd, [$base, offset]`**
- Operation: load unsigned byte.
- One byte is read from memory, zero-extended to 32 bits, then put in general purpose register **`$rd`**.

5.9.6 `lh`

- Syntax: **`lh $rd, [$base, offset]`**
- Operation: load signed halfword.
- Two bytes are read from memory, sign-extended to 32-bits, then put in general purpose register **`$rd`**.

5.9.7 `lhu`

- Syntax: **`lhu $rd, [$base, offset]`**
- Operation: load unsigned halfword.
- Two bytes are read from memory, zero-extended to 32-bits, then put in general purpose register **`$rd`**.

5.9.8 `lw`

- Syntax: **`lw $rd, [$base, offset]`**
- Operation: load word.
- Four bytes are read from memory, then put in general purpose register **`$rd`**.

5.9.9 `ld`

- Syntax: **`ld #rd, [$base, offset]`**
- Operation: loads float.
- Four bytes are read from memory, interpeted as an IEEE 754 binary32 type, extended to binary64, then placed in floating-point register **`#rd`**.

5.9.10 `ldl`

- Syntax: **`ldl #rd, [$base, offset]`**
- Operation: load lower double-half.
- Four bytes are read from memory, then put in the lower 32-bits of floating-point register **`#rd`**. This operation is raw, leaving the upper bits untouched.
- Condition flags are updated for **`#rd`** as a whole.

5.9.11 `ldu`

- Syntax: **`ldu #rd, [$base, offset]`**
- Operation: load upper double-half.
- Four bytes are read from memory, then put in the upper 32-bits of floating-point register **`#rd`**. This operation is raw, leaving the lower bits untouched.
- Condition flags are updated for **`#rd`** as a whole.

5.9.12 `sb`

- Syntax: **`sb $rs, [$base, offset]`**
- Operation: store byte.
- The low 8 bits of **`$rs`** are written to the calculated memory address.

5.9.13 `sh`

- Syntax: **`sh $rs, [$base, offset]`**
- Operation: store halfword.
- The low 16 bits of **`$rs`** are written to the calculated memory address memory using big-endian byte order.

5.9.14 `sw`

- Syntax: **`sw $rs, [$base, offset]`**
- Operation: store word.
- The full 32-bit value in **`$rs`** is written to the calculated memory address using big-endian byte order.

5.9.15 `sd`

- Syntax: **`sd #rs, [$base, offset]`**
- Operation: store an IEEE-754 binary32 floating-point value.
- The binary64 value held in floating-point register **`#rs`** is converted to IEEE-754 binary32 and the resulting 32-bit representation is written to the calculated memory address using big-endian byte order.
- The conversion performed by **`sd`** is blind: **`sd`** does not report whether precision or range was lost during conversion and does not update condition flags.
- Software that requires assurance that the binary64 source can be represented appropriately as binary32 should execute **`disf`** before performing the store.

5.9.16 `sdl`

- Syntax: **`sdl #rs, [$base, offset]`**
- Operation: store lower binary64.
- The lower 32-bit value from floating-point register **`#rs`** is written to the calculated memory address using big-endian byte order.

5.9.17 `sdu`

- Syntax: **`sdu #rs, [$base, offset]`**
- Operation: store upper binary64.
- The upper 32-bit value from floating-point register **`#rs`** is written to the calculated memory address using big-endian byte order.

5.10 Constant Construction Instructions

VIV-32 instructions are fixed-width 32-bit words. A single instruction cannot encode every possible 32-bit constant together with an opcode and register fields. Larger constants and addresses are therefore constructed using one or more instructions.

Constants construction instructions set the conditions register **%sr** as per normal.

5.10.1 lui

- Syntax: **lui $rd, imm16**
- Operation: load upper immediate, **rd** ← **imm16 << 16**
- The 16-bit immediate is placed in bits 31–16 of **$rd**, with bits 15–0 of the result set to zero.

5.10.2 lli

- Syntax: **lli $rd, imm16**
- Operation: load lower immediate, **rd** ← **imm16 & 0xFFFF**
- The 16-bit immediate is zero-extended and written to **$rd**.

5.10.3 lhi

- Syntax: **lhi $rd, imm16**
- Operation: load upper immediate, preserving lower bytes,
  $rd ← **($rd & 0xFFFF) | (imm16 << 16)**

5.10.4 Full-Width Constants, li and la

The assembler provides pseudo instructions **li** and **la** to load 32-bit values and addresses respectively.

5.11 Inter-File Transfer Instructions

5.11.1 Floating-Point Rounding

VIV-32 v0.2 does not provide a programmable floating-point rounding-mode control register.

Unless explicitly specified otherwise by an instruction, floating-point arithmetic and floating-point conversions use IEEE-754 round-to-nearest, ties-to-even.

This rounding mode applies to operations including:

- IEEE-754 binary64 arithmetic;
- binary64-to-binary32 conversion;
- integer-to-binary64 conversion where rounding is required;
- binary64-to-integer conversion.

Where a conversion or arithmetic operation cannot represent the exact mathematical result in its destination format, **X** is set when rounding is required, subject to the instruction-specific flag rules.

5.11.2 `disf`

- Syntax: **`disf #ra`**
- Operation: tests the result of converting the IEEE-754 binary64 value in **`#ra`** to IEEE-754 binary32 without storing the converted value.
- The source floating-point register is not modified.
- The condition flags describe the binary32 conversion that would be performed by **`dtof`** or **`sd`**:
  - **`N`** is set if the converted value is numerically negative.
  - **`Z`** is set if the converted value is `+0.0` or `-0.0`.
  - **`I`** is set if the converted value is positive or negative **`infinity`**.
  - **`U`** is set if conversion enters the binary32 underflow range.
  - **`X`** is set if conversion to binary32 requires rounding or otherwise loses precision.
  - **`E`** is set only if the conversion is invalid.
  - **`C`** and **`V`** are cleared.
- IEEE-754 **`NaN`** values are considered representable binary32 values and may therefore be tested by **`disf`**; the presence of a **`NaN`** does not by itself make the conversion invalid.
- No converted value is written to a register.

5.11.3 `utod`

- Syntax: **`utod #rd, $rs`**
- Operation: **`#rd`** ← **`FLOAT($rs)`**
- Loads numerical value of unsigned general-purpose register **`$rs`** into floating-point register **`#rd`** as binary64, maintaining numerical value.

5.11.4 `itod`

- Syntax: **`itod #rd, $rs`**
- Operation: **`#rd`** ← **`FLOAT($rs)`**
- Loads numerical value of signed general-purpose register **`$rs`** into floating-point register **`#rd`** as binary64, maintaining numerical value.

5.11.5 `ftod`

- Syntax: **`ftod #rd, $rs`**
- Operation: **`#rd`** ← **`BINARY64($rs)`**
- Loads binary32 value of general-purpose register **`$rs`** into floating-point register **`#rd`** as binary64.

5.11.6 `dtou`

- Syntax: **`dtou $rd, #rs`**
- Operation: **`$rd ← UNSIGNED(#rs)`**
- Converts the IEEE-754 binary64 numerical value in **`#rs`** to an unsigned 32-bit integer using the architectural rounding mode.
- On successful conversion:
  - The converted value is written to **`$rd`**.
  - **`N`** is set from bit 31 of the resulting integer.
  - **`Z`** is set if the resulting integer is zero.
  - **`C`**, **`V`**, and **`E`** are cleared.
  - **`I`** and **`U`** are cleared.
  - **`X`** is set if rounding was required to produce the integer result; otherwise it is cleared.
- If the source is **`NaN`**, positive or negative infinity, or cannot be represented as an unsigned 32-bit integer after rounding:
  - **`E`** is set.
  - **`N`**, **`Z`**, **`C`**, **`V`**, **`I`**, **`U`**, and **`X`** are cleared.
  - **`$rd`** is left unchanged.

5.11.7 `dtoi`

- Syntax: **`dtoi $rd, #rs`**
- Operation: **`$rd ← SIGNED(#rs)`**
- Converts the IEEE-754 binary64 numerical value in **`#rs`** to a signed 32-bit two's-complement integer using the architectural rounding mode.
- On successful conversion:
  - The converted value is written to **`$rd`**.
  - **`N`** is set if the resulting integer is negative.
  - **`Z`** is set if the resulting integer is zero.
  - **`C`**, **`V`**, and **`E`** are cleared.
  - **`I`** and **`U`** are cleared.
  - **`X`** is set if rounding was required to produce the integer result; otherwise it is cleared.
- If the source is **`NaN`**, positive or negative infinity, or cannot be represented as a signed 32-bit integer after rounding:
  - **`E`** is set.
  - **`N`**, **`Z`**, **`C`**, **`V`**, **`I`**, **`U`**, and **`X`** are cleared.
  - **`$rd`** is left unchanged.

5.11.8 `dtof`

- Syntax: **`dtof $rd, #rs`**
- Operation: **`$rd`** ← **`BINARY32(#rs)`**
- Converts the IEEE-754 binary64 value in **`#rs`** to IEEE-754 binary32 using the architectural rounding mode and writes the resulting 32-bit representation to **`$rd`**.
- The condition flags describe the resulting binary32 value:
  - **`N`** is set if the converted value is numerically negative.
  - **`Z`** is set if the converted value is `+0.0` or `-0.0`.
  - **`I`** is set if the converted value is positive or negative infinity.
  - **`U`** is set if conversion enters the binary32 underflow range.
  - **`X`** is set if conversion requires rounding or otherwise loses precision.
  - **`E`** is set only if the conversion is invalid.
  - **`C`** and **`V`** are cleared.
- IEEE-754 **`NaN`** values are representable as binary32 values and do not by themselves make the conversion invalid.
- If the conversion is invalid, **`$rd`** is left unchanged.

5.12 Branch Instructions

Branch instructions perform conditional PC-relative control transfer. VIV-32 defines two branch families:

- register-comparison branches, which compare two registers directly;
- flag-based branches, which test the existing condition flags in **`%sr`**.

5.12.1 Branch Target Calculation

- Branch offsets are signed PC-relative offsets.
- Branch offsets are measured in 32-bit instruction words.
- The encoded offset is shifted left by two bits to form a byte offset.
- The branch target is relative to the address of the instruction following the branch.
- Branch targets must be 4-byte aligned.
- Branch instructions do not update condition flags.

5.12.2 Register-Comparison Branches

Register-comparison branches compare **`ra`** and **`rb`** directly. They do not depend on the current condition flags, and they do not update the condition flags. The jump target is calculated in instruction words:

```
target = %pc + 4 + SEXT(imm14) << 2
```

That is, the target is relative to the NEXT instruction, not the executed instruction.

- **`imm14`** can be a label or a value.

- Unsigned comparisons are only valid for general purpose registers.
- Floating-point comparisons use IEEE-754 binary64 numerical comparison semantics.
- If either floating-point operand is `NaN`, the comparison is unordered. For unordered comparisons, `bd.eq`, `bd.lt`, `bd.le`, `bd.gt`, and `bd.ge` do not branch; `bd.ne` branches.
- Register-comparison branches do not modify the condition flags, including for unordered floating-point comparisons.

5.12.3 `b.eq, bd.eq`

- Syntax: `b.eq $ra, $rb, imm14 | bd.eq #ra, #rb, imm14`
- Operation: branch if `ra = rb`.
- For general purpose registers, this is bitwise equivalence. For floating-point register, this is numerical equivalence.

5.12.4 `b.ne, bd.ne`

- Syntax: `b.ne $ra, $rb, imm14 | bd.ne #ra, #rb, imm14`
- Operation: branch if `ra` $\neq$ `rb`.
- For general purpose registers, this is bitwise inequivalence. For floating-point register, this is numerical inequivalence.

5.12.5 `b.lt, bd.lt`

- Syntax: `b.lt $ra, $rb, imm14 | bd.lt #ra, #rb, imm14`
- Operation: branch if `ra < rb`, treating both operands as signed numbers.

5.12.6 `b.le, bd.le`

- Syntax: `b.le $ra, $rb, imm14 | bd.le #ra, #rb, imm14`
- Operation: branch if `ra` $\leq$ `rb`, treating both operands as signed numbers.

5.12.7 `b.gt, bd.gt`

- Syntax: `b.gt $ra, $rb, imm14 | bd.gt #ra, #rb, imm14`
- Operation: branch if `ra > rb`, treating both operands as signed numbers.

5.12.8 `b.ge, bd.ge`

- Syntax: `b.ge $ra, $rb, imm14 | bd.ge #ra, #rb, imm14`
- Operation: branch if `ra` $\geq$ `rb`, treating both operands as signed numbers.

5.12.9 `b.ltu`

- Syntax: `b.ltu $ra, $rb, imm14`
- Operation: branch if `ra < rb`, treating both operands as unsigned 32-bit integers.

5.12.10 `b.leu`

- Syntax: `b.leu $ra, $rb, imm14`
- Operation: branch if `ra` $\leq$ `rb`, treating both operands as unsigned 32-bit integers.

5.12.11 `b.gtu`

- Syntax: `b.gtu $ra, $rb, imm14`
- Operation: branch if `ra > rb`, treating both operands as unsigned 32-bit integers.

5.12.12 `b.geu`

- Syntax: `b.geu $ra, $rb, imm14`
- Operation: branch if `ra` $\geq$ `rb`, treating both operands as unsigned 32-bit integers.

5.12.13 Flag-Based Branches

Flag-based branches test a single current condition flag in the status register `%sr`. They do not perform a comparison, do not interpret the flags as signed, unsigned, or floating-point state, and do not update any flags.

The jump target is calculated in instruction words:

```
target = %pc + 4 + SEXT(imm22) << 2
```

That is, the target is relative to the instruction following the branch. `imm22` may be a label or a value.

5.12.14 `bf.ns`

- Syntax: `bf.ns imm22`
- Operation: branch if `N` is set.

5.12.15 `bf.nc`

- Syntax: `bf.nc imm22`
- Operation: branch if `N` is clear.

5.12.16 `bf.zs`

- Syntax: `bf.zs imm22`
- Operation: branch if `Z` is set.

5.12.17 `bf.zc`

- Syntax: `bf.zc imm22`
- Operation: branch if `Z` is clear.

5.12.18 `bf.cs`

- Syntax: `bf.cs imm22`
- Operation: branch if `C` is set.

5.12.19 `bf.cc`

- Syntax: **`bf.cc imm22`**
- Operation: branch if **`C`** is clear.

5.12.20 `bf.vs`

- Syntax: **`bf.vs imm22`**
- Operation: branch if **`V`** is set.

5.12.21 `bf.vc`

- Syntax: **`bf.vc imm22`**
- Operation: branch if **`V`** is clear.

5.12.22 `bf.es`

- Syntax: **`bf.es imm22`**
- Operation: branch if **`E`** is set.

5.12.23 `bf.ec`

- Syntax: **`bf.ec imm22`**
- Operation: branch if **`E`** is clear.

5.12.24 `bf.is`

- Syntax: **`bf.is imm22`**
- Operation: branch if **`I`** is set.

5.12.25 `bf.ic`

- Syntax: **`bf.ic imm22`**
- Operation: branch if **`I`** is clear.

5.12.26 `bf.us`

- Syntax: **`bf.us imm22`**
- Operation: branch if **`U`** is set.

5.12.27 `bf.uc`

- Syntax: **`bf.uc imm22`**
- Operation: branch if **`U`** is clear.

5.12.28 `bf.xs`

- Syntax: **`bf.xs imm22`**
- Operation: branch if **`X`** is set.

5.12.29 `bf.xc`

- Syntax: `bf.xc imm22`
- Operation: branch if `X` is clear.

5.13 Jump and Call Instructions

Jump and call instructions perform unconditional control transfer. Unlike branch instructions, jump and call instructions do not test condition flags or compare registers.

5.13.1 Jump Target Rules

- Jump and call targets are 32-bit instruction addresses.
- The encoded offset is measured in 32-bit instruction words.
- The encoded offset is shifted left by two bits to form a byte offset.
- The target is relative to the address of the following instruction.
- Jump and call targets must be 4-byte aligned. This is ensured by the calculation:
  `%pc ← %pc + 4 + SEXT(imm26) << 2`
- Jump and call instructions do not update condition flags.
- `imm26` can be a value or label.

5.13.2 `jmp`

- Syntax: `jmp imm26`
- Operation: unconditional jump.

5.13.3 `call`

- Syntax: `call imm26`
- Operation: unconditional jump with updated link register `%lr` (`$15`). The return address is the address of the instruction following the `call`.
  `$15 ← %pc + 4`

5.13.4 `jr`

- Syntax: `jr $target`
- Operation: register-indirect unconditional jump, `%pc ← $target`
- The `target` register contains the destination address.
- The destination address must be 4-byte aligned. Causes a `MisalignedFetchException` if the `target` address is not word-aligned.

5.13.5 `jalr`

- Syntax: `jalr $rd, $target`
- Operation: register-indirect call, `%pc` ← `UNSIGNED(target)`
- The `target` register contains the destination address.
- The destination address must be 4-byte aligned. Causes a `MisalignedFetchException` if the `target` address is not word-aligned.
- The return address is written to `rd`, `rd` ← `%pc + 4`

5.13.6 `ret`

- `ret` is an assembler pseudo-instruction, not a distinct architectural instruction in VIV-32 v0.2.
- Syntax: `ret`
- Expansion: `jr $15`
- Operation: return to the address held in the link register.
- No condition flags are updated.

5.13.7 Nested Calls

- A call instruction overwrites the link register.
- A function that calls another function must save its incoming link register value if it needs to return to its own caller, e.g.:

```
push $lr
call func
pop $lr
```

5.14 System and Control Instructions

System and control instructions provide access to processor control state, exception handling, interrupt control, and execution control.

5.14.1 Control Registers

- VIV-32 defines a small set of architectural control registers.
- Control registers are accessed using `mrs` and `msr`.
- Access permissions are defined by the privilege model. In VIV-32 v0.2, only machine mode is defined.

Name	Meaning	Description
`%pc`	Program counter	Address of the currently executing instruction
`%sr`	Status register	Condition flags and interrupt-enable state
`%epc`	Exception program counter	Continuation address saved on exception entry
`%esr`	Exception status register	Status register saved on exception entry
`%ecause`	Exception cause	Cause code saved on exception entry
`%edata`	Exception data	Exception-specific data saved on exception entry
`%evbase`	Exception vector base	Base address for exception and interrupt vectors

For the purposes of instruction semantics, `%pc` contains the address of the instruction currently being executed. References to `%pc + 4` therefore refer to the address of the following sequential instruction.

At an architectural instruction boundary, before the next instruction begins execution, `%pc` identifies the next instruction to be executed.

5.14.2 `nop`

- Syntax: `nop`
- Operation: no operation.
- Encoding: `0x00000000`.
- No general-purpose register is modified.
- No control register is modified.
- No condition flags are updated.
- Execution continues at the next instruction.

5.14.3 `halt`

- Syntax: `halt`
- Operation: halt instruction execution.
- The processor stops executing instructions.
- The processor remains halted until reset or platform-defined external action.
- No condition flags are updated.

5.14.4 `trap`

- Syntax: `trap imm12`
- Operation: raise a software trap exception.
- The immediate operand is encoded as the X-type `imm12` payload.
- On trap entry, `%ecause` receives `SoftwareTrap`.
- On trap entry, `%edata` receives `ZEXT(imm12)`.
- On trap entry, `%epc` receives the address of the instruction following the `trap` instruction.
- The exception entry sequence is defined in the exceptions and interrupts chapter.
- No condition flags are updated.

5.14.5 `syscall`

- Syntax: **`syscall`**
- Operation: raise a system call exception.
- **`syscall`** is a distinct architectural instruction.
- **`syscall`** may share exception-entry machinery with **`trap`**.
- On syscall entry, **`%epc`** receives the address of the instruction following the **`syscall`**.
- On syscall entry, **`%ecause`** receives the system call exception code.
- The system call number and arguments are passed according to the ABI.
- The exception entry sequence is defined in the exceptions and interrupts chapter.
- No condition flags are updated.

5.14.6 `iret`

- Syntax: **`iret`**
- Operation: return from an exception or interrupt.
- The program counter is restored from **`%epc`**.
- The processor status register is restored from **`%esr`**.
- Operation:

$$\texttt{\%pc} \leftarrow \texttt{\%epc}$$

$$\texttt{\%sr} \leftarrow \texttt{\%esr}$$

- All condition flags and the interrupt-enable state are restored as part of the **`%sr`** restoration.

5.14.7 `ei`

- Syntax: **`ei`**
- Operation: enable maskable interrupts.
- The interrupt-enable bit in **`%sr`** is set.
- No general-purpose register is modified.
- No condition flags are updated.

5.14.8 `di`

- Syntax: **`di`**
- Operation: disable maskable interrupts.
- The interrupt-enable bit in **`%sr`** is cleared.
- No general-purpose register is modified.
- No condition flags are updated.

5.14.9 `rdpc`

- Syntax: `rdpc $rd`
- Operation: read program counter.
- The address of the `rdpc` instruction is written to `rd`.
- Operation: `rd` ← `%pc`
- No condition flags are updated.

5.14.10 `mrs`

- Syntax: `mrs $rd, %cr`
- Operation: move from control register.
- The value of control register `cr` is written to `rd`.
- No condition flags are updated.

5.14.11 `msr`

- Syntax: `msr %cr, $rs`
- Operation: move to control register.
- The value of `rs` is written to control register `cr`. Reserved bits are not updated.
- Writes to control registers may have immediate architectural effects.
- `%pc` is read-only through the control-register interface.
- Executing `msr %pc, $rs` raises an `IllegalInstructionException`. The program counter is not modified by the control-register write.
- No condition flags are updated, except when writing `%sr`.

Writing to `%evbase`:

- Writing `%evbase` requires the source value to be 16-byte aligned.
- If the source value is not 16-byte aligned, `%evbase` is left unchanged and the processor enters the halted state.
- This condition does not raise an exception.

5.15 Pseudo-Instructions

Pseudo-instructions are assembler conveniences. They do not necessarily correspond to distinct architectural instruction encodings. A conforming assembler may expand pseudo-instructions into one or more real VIV-32 instructions.

5.15.1 Pseudo-Instruction Policy

- Pseudo-instructions are not part of the architectural instruction encoding unless explicitly stated otherwise.
- Pseudo-instructions may expand differently depending on immediate size, relocation requirements, or target address range.
- The assembler is responsible for selecting a correct expansion.
- The exact expansion rules are defined by the assembler and toolchain specification.
- Pseudo-instructions do not define independent architectural side effects beyond those produced by their expansion into real VIV-32 instructions.
- Unless a pseudo-instruction definition explicitly constrains the expansion, condition flags, control-register effects, exceptions, and other observable behaviour are those of the selected instruction sequence.
- A conforming assembler must preserve the functional meaning of the pseudo-instruction, but different valid expansions may differ in incidental architectural side effects where this specification does not require otherwise.

5.15.2 `ldd`

- Syntax: **`ldd #rd, [$base, imm15]`**
- Operation: loads a binary64 value from the calculated memory address, as per the load/store memory address algorithm, into floating-point register **`rd`**. Loads in big-endian order, with the upper double-half at the calculated address, and the lower double-half **`address + 4`**.

5.15.3 `sdd`

- Syntax: **`sdd #rs, [$base, imm15]`**
- Operation: stores a binary64 value from floating-point register **`rs`** into the calculated memory address, as per the load/store memory address algorithm. Stores in big-endian order, with the upper double-half at the calculated address, and the lower double-half at **`address + 4`**.

5.15.4 `li`

- Syntax: **`li $rd, imm32`**
- Operation: load a 32-bit immediate value into **`rd`**.
- The assembler expands **`li`** into one or more constant-construction instructions, e.g.:

  ```
  lli rd, imm32 & 0xFFFF
  lhi rd, (imm32 >> 16) & 0xFFFF
  ```

5.15.5 `la`

- Syntax: **`la $rd, symbol`**
- Operation: load the address of **`symbol`** into **`rd`**.
- **`symbol`** **MUST** be a label.

- The assembler expands `la` into relocation-aware address construction instructions. The final address will be resolved by the linker.

5.15.6 `mov, movd`

- Syntax: `mov $rd, $rs | movd #rd, #rs`
- Operation: copy the value of `rs` into `rd`, `rd` ← `rs`
- The internal construction ensures flags are updated accordingly.

5.15.7 `clr, clrd`

- Syntax: `clr $rd | clrd #rd`
- Operation: write zero to `rd`, `rd` ← `0`
- The internal construction ensure flags are updated accordingly.

5.15.8 `ret`

See definition under Jump and Call Instructions.

5.15.9 `push, pushd`

- Syntax: `push $rs | pushd #rs`
- Operation: push a register value onto the stack.
- The stack grows downward.
- The assembler expands the pseudo-instruction into the appropriate stack-pointer adjustment and store instruction sequence.
- Preservation of ABI stack-alignment requirements, including alignment at function-call boundaries, is the responsibility of the toolchain and generated instruction sequence rather than the architectural `push` or `pushd` pseudo-instruction itself.

5.15.10 `pop, popd`

- Syntax: `pop $rd | popd #rd`
- Operation: pops a register value from the stack.
- Care must be taken by the programmer to ensure this is used in conjunction with the appropriate `push, pushd`. If the stack is not word-aligned, this will cause a `MisalignedAccessException`.

5.16 Reserved Instructions

- Instruction encodings not defined by VIV-32 v0.2 are reserved.
- Reserved encodings must not be generated by conforming assemblers.
- Reserved encodings are reserved for future architectural extensions, implementation-specific features, or debugging facilities.
- Executing a reserved encoding raises an illegal-instruction exception.

- On an illegal-instruction exception, **%epc** receives the continuation address, with the offending address placed into **%edata**.
- On an illegal-instruction exception, **%ecause** receives the illegal-instruction exception code.

5.17 Instruction Set Status

- The VIV-32 v0.2 instruction set is defined by this specification. Incompatible future architectural changes require a new architecture version.
- Instruction semantics in this chapter are architectural unless explicitly described as pseudo-instructions.
- Opcode assignments are defined separately by the opcode map.
- Assembler syntax and pseudo-instruction expansion rules are defined by the toolchain specification.

Chapter 6

Exceptions and Interrupts

6.1 Exception and Interrupt Model

VIV-32 uses a single exception and interrupt entry mechanism. Synchronous exceptions, software traps, system calls, and hardware interrupts all transfer control through the exception vector mechanism.

- The current processor status is saved to **%esr**.
- The continuation address is saved to **%epc**.
- The exception cause code is written to **%ecause**.
- Exception-specific data is written to **%edata**.
- The interrupt-enable bit in **%sr** is cleared.
- Control transfers to a vector address derived from **%evbase** and **%ecause**.

VIV-32 v0.2 defines machine mode only. User mode, supervisor mode, privilege transitions, nested privilege stacks, and memory protection are reserved for future versions.

6.2 Exception Causes

The **%ecause** control register records the cause of the most recent exception or interrupt.

ecause value	Cause	Description
0x00	**RESET**	Reset entry
0x01	**IllegalInstruction**	Undefined, reserved or otherwise invalid instruction
0x02	**MisalignedFetchException**	Instruction fetch or control transfer to a non-word-aligned ad
0x03	**MisalignedAccessException**	Misaligned halfword, word, or double-half load/store
0x04	**SoftwareTrap**	Software-generated exception raised by the **trap** instruction;
0x05	**SystemCall**	**syscall** instruction
0x06	**Interrupt**	Bus-attached device interrupt
0x07	**BusError**	
0x08-0xFF	Reserved	Reserved for future architectural or platform causes

Cause values above **0xFF** are reserved in VIV-32 v0.2. Future versions may define additional architectural, platform, or implementation-specific exception causes.

6.3 Exception Vector Calculation

The **%evbase** control register contains the base address of the exception vector table. Each exception cause has a 16-byte vector slot.

```
vector_address ← %evbase + (%ecause << 4)
```

The **%evbase** register must contain a 16-byte-aligned address.

An attempt to write a value to **%evbase** whose low four bits are non-zero is an unrecoverable architectural error. The attempted value is not written to **%evbase**, and the processor enters the halted state. No exception is raised, because exception entry requires a valid exception-vector base address. Execution remains halted until reset or platform-defined external action.

- Each vector slot is 16 bytes.
- Each vector slot can contain up to four 32-bit instructions.
- A vector slot may contain a short handler directly.
- A vector slot may contain a jump to a longer handler elsewhere.

6.4 Exception Entry Sequence

When an exception or interrupt is taken, the processor performs the following architectural actions:

1. Determine the exception cause.
2. Save the current processor status register to **%esr**.
3. Clear the interrupt-enable bit in **%sr**.
4. Save the current continuation address to **%epc**.
5. Write the exception cause code to **%ecause**.
6. Write exception-specific data to **%edata**.
7. Set the program counter to **%evbase + (%ecause << 4)**.

The value saved in **%esr** is the complete pre-exception value of **%sr**, including the interrupt-enable bit and condition flags.
Clearing the interrupt-enable bit prevents maskable interrupts from interrupting the initial exception handler sequence. Software may re-enable interrupts explicitly after saving any required state.

6.5 Exception Program Counter

The **%epc** control register contains the continuation address saved when an exception or interrupt is taken.

The continuation address is the address at which execution will resume if the handler returns using **iret** without modifying **%epc**.

For exceptions raised during execution of an instruction, the continuation address is normally the address of the following sequential instruction:

`%epc` ← `%pc + 4`

This includes software traps, system calls, illegal instructions, misaligned data accesses, and other exceptions detected while executing an instruction.

For an interrupt taken between instructions, **`%epc`** receives the address of the next instruction that would otherwise have executed.

For an exception raised while fetching an instruction, **`%epc`** receives the address from which instruction fetch was attempted. Returning without modifying **`%epc`** therefore retries the failed fetch.

Exception-specific addresses or other diagnostic information are recorded separately in **`%edata`**.

An exception handler may modify **`%epc`** before executing **`iret`** in order to retry, skip, or redirect execution.

6.6 Exception Data Register

The **`%edata`** control register records data associated with the most recent exception or interrupt. The interpretation of **`%edata`** depends on the exception cause.

Event type	**`%edata` value**	**Meaning**
IllegalInstruction	Instruction address	Address of the invalid instruction
MisalignedFetchException	Instruction address	Misaligned fetch address or control-transfer target
MisalignedAccessException	Effective address	Address of the misaligned memory access
SoftwareTrap	Trap immediate	Encoded software-trap data
SystemCall	`0`	No architecture-defined exception data
Interrupt	Platform interrupt data	Interrupt-source information supplied by the platform
BusError	Transaction address	Address associated with the failed bus transaction

The interpretation of platform interrupt data is defined by the platform specification.

6.7 Status Register Interrupt State

The **`%sr`** control register contains the architectural condition flags and interrupt-control state. On exception or interrupt entry, the current value of **`%sr`** is copied to **`%esr`** before **`%sr.IE`** is cleared. **`%esr`** therefore contains the complete processor status that existed immediately before exception entry, including all condition flags and the previous interrupt-enable state. The active **`%sr`** may then be modified by the exception handler without destroying the saved pre-exception status.

Bit	Name	Meaning
`0`	`N`	Negative flag
`1`	`Z`	Zero flag
`2`	`C`	Carry flag
`3`	`V`	Overflow flag
`4`	`I`	Infinity flag
`5`	`U`	Underflow flag
`6`	`X`	Inexact flag
`7`	`E`	Arithmetic error flag
`8`	`IE`	Interrupt enable bit
`9-31`	reserved	Reserved, must be zero in VIV-32 v0.2

The **%esr** register uses the same bit layout as **%sr**. Bits 0–8 of **%esr** correspond directly to the defined **%sr** fields. Bits 9–31 are reserved and read as zero. Writes to reserved **%esr** bits are ignored. Consequently, restoring **%sr** from **%esr** using **iret** cannot introduce non-zero values into reserved **%sr** bits.
When **IE** is set, maskable interrupts may be taken. When **IE** is clear, maskable interrupts are held pending or ignored according to platform interrupt-controller behaviour.

- The **ei** instruction sets **%sr.IE**.
- The **di** instruction clears **%sr.IE**.
- Exception and interrupt entry clears **%sr.IE**.
- The **iret** instruction restores **%pc** from **%epc** and restores **%sr** from **%esr**.

6.8 Interrupts

Hardware interrupts use the common exception-entry mechanism.

For an interrupt:

- **%ecause** is set to **0x06**.
- **%edata** receives platform-defined interrupt-source information.

The exact interrupt sources, priorities, pending-bit behaviour, acknowledgement mechanism, and interpretation of **%edata** are defined by the platform specification.

6.9 Software Trap

The **trap** instruction raises a **SoftwareTrap** exception.

For a software trap:

- **%ecause** is set to **0x04**.
- **%edata** receives the zero-extended 12-bit immediate operand encoded by the **trap** instruction.

The interpretation of the trap data is defined by the execution environment.

6.10 System Call

The **syscall** instruction raises a **SystemCall** exception.

For a system call:

- **%ecause** is set to **0x05**.
- **%edata** is set to zero.

The system-call number and arguments are passed according to the VIV-32 ABI. The system-call ABI and execution-environment semantics are specified separately.

6.11 Illegal Instruction

An `IllegalInstructionException` is raised when the processor encounters an undefined or reserved instruction encoding, or executes an instruction whose encoded operation is architecturally invalid.

For an illegal instruction:

- `%ecause` is set to `0x01`.
- `%edata` receives the address of the invalid instruction.

6.12 Misaligned Access Exceptions

A `MisalignedFetchException` is raised when the processor would transfer instruction execution to an address that is not word-aligned.

This may occur either:

- while attempting to fetch an instruction from a misaligned address; or
- while executing a control-transfer instruction whose calculated or register-supplied target address is not word-aligned.

For a `MisalignedFetchException`:

- `%ecause` is set to `0x02`.
- `%edata` receives the misaligned instruction address.

The value saved in `%epc` follows the normal continuation-address rules:

- for an exception detected while executing a control-transfer instruction, `%epc` receives the address of the following sequential instruction;
- for an exception detected while fetching an instruction, `%epc` receives the attempted fetch address.

A `MisalignedAccessException` is raised when a halfword, word, or double-half load or store uses an effective address that does not satisfy its required alignment.

For a misaligned data access:

- `%ecause` is set to `0x03`.
- `%edata` receives the effective address of the misaligned access.

Byte loads and stores never raise `MisalignedAccessException`.

6.13 Bus Error

A `BusError` exception is raised when an instruction fetch, load, or store cannot be completed because the addressed bus transaction is unavailable or unsupported.

For a bus error:

- `%ecause` is set to `0x07`.
- `%edata` receives the address associated with the failed bus transaction.

6.14 Exception Return

Exception and interrupt handlers return using the **iret** instruction.

On execution of **iret**:

- **%pc** is restored from **%epc**.
- **%sr** is restored from **%esr**.

Restoring **%sr** restores the complete pre-exception processor status, including the condition flags and interrupt-enable state.

Consequently, **iret** does not unconditionally enable interrupts. If interrupts were enabled before the exception was taken, they are enabled again on return; if they were disabled, they remain disabled.

6.15 Nested Exceptions and Interrupts

Exception and interrupt entry clears **%sr.IE**, preventing maskable interrupts from being taken while the initial handler sequence executes.

Software may explicitly re-enable interrupts from within a handler, permitting nested or reentrant interrupt handling.

The exception-state control registers are not stacked automatically. A subsequent exception or interrupt overwrites:

- **%epc**;
- **%esr**;
- **%ecause**;
- **%edata**.

Software that permits nested exceptions or interrupts must preserve any exception state required by the interrupted handler before re-enabling interrupts.

The storage location, ordering, nesting depth, and reentrancy mechanisms used for this preservation are software policy and are outside the scope of the base architecture.

Chapter 7

Reset and Boot

7.1 Reset Model

Reset places the processor into a known architectural state and begins instruction execution from the reset vector.

VIV-32 v0.2 defines only machine-mode reset. User-mode reset, supervisor-mode reset, secure boot, and multi-stage privilege transitions are reserved for future versions.

7.2 Reset Vector

On reset, the program counter is set to the architectural reset vector.

```
%pc = 0x00000000
```

The reset vector address must be 4-byte aligned. The platform must ensure that executable reset code is available at the reset vector address.
A platform may implement the reset vector using ROM, RAM preloaded by an emulator, memory-mapped boot firmware, or another platform-defined boot mechanism.

7.3 Reset State

After reset, architectural state is initialized as follows:

State	Reset value	Notes
`PC`	`0x00000000`	Reset vector
`$0`	`0x00000000`	Hardwired zero
`$1-$15`	Undefined	Software must initialize before use
`#0-#7`	Undefined	Software must initialize before use
`%sr.N`	`0`	Negative flag cleared
`%sr.Z`	`0`	Zero flag cleared
`%sr.C`	`0`	Carry flag cleared
`%sr.V`	`0`	Overflow flag cleared
`%sr.I`	`0`	Infinity flag cleared
`%sr.U`	`0`	Underflow flag cleared
`%sr.X`	`0`	Inexact flag cleared
`%sr.E`	`0`	Arithmetic error flag cleared
`%sr.IE`	`0`	Interrupts disabled
`%epc`	Undefined	No exception has been taken
`%esr`	Undefined	No exception status has been saved
`%ecause`	`0x00`	Reset cause
`%edata`	`0x00000000`	No exception data
`%evbase`	Platform-defined	Must be 16-byte aligned; software must initialize before exception handling is

Any platform-defined reset value of `%evbase` must satisfy the architectural 16-byte alignment requirement. General-purpose registers other than `r0` are undefined after reset. Boot code must initialize `$sp`, global pointers, frame pointers, and any other registers required by the ABI or execution environment.

7.4 Reset Cause

Reset is represented architecturally by exception cause value `0x00`. After reset, `%ecause` contains `0x00`.
Reset does not require the processor to enter through the ordinary exception vector calculation. On reset, execution begins directly at the reset vector.

7.5 Initial Interrupt State

Interrupts are disabled after reset.

- The `IE` bit in `sr` is cleared.
- Timer interrupts are not taken until software enables interrupts.
- External interrupts are not taken until software enables interrupts.
- Pending interrupt state, if any, is platform-defined.

Boot firmware should initialize `evbase`, configure platform interrupt devices, install exception handlers, and then enable interrupts using `ei` when ready.

7.6 Initial Memory State

The contents of RAM after reset are platform-defined.

- RAM may contain undefined data.

- ROM or boot firmware contents are platform-defined.
- Memory-mapped device reset state is defined by the platform specification.
- The base ISA does not require memory to be cleared on reset.

Software must not assume that RAM is zero-filled unless the platform specification explicitly guarantees it.

7.7 Boot Responsibilities

The base architecture defines only the architectural reset state. Platform firmware or emulator boot code is responsible for preparing the execution environment.

Typical boot responsibilities include:

- initializing the stack pointer;
- initializing **`evbase`**;
- installing exception and interrupt handlers;
- clearing or initializing the **`.bss`** region;
- initializing global data;
- configuring memory-mapped devices;
- configuring timer and external interrupt sources;
- enabling interrupts when the system is ready;
- transferring control to firmware, a monitor, a kernel, or a standalone program.

7.8 Boot Sequence Example

A simple platform boot sequence may perform the following operations:

1. Begin execution at `0x00000000`.
2. Disable interrupts using **`di`**.
3. Initialize **`sp`**.
4. Initialize **`evbase`**.
5. Install or verify exception vector contents.
6. Initialize memory and global data.
7. Configure platform devices.
8. Enable interrupts using **`ei`**, if required.
9. Transfer control to the main program.

This sequence is illustrative. The exact boot sequence is defined by the platform and execution environment.

Chapter 8

Implementation Notes